BATPING (Bidirectional Acoustics Traveltime Pulser for INdex Gradients) 2026

P400 lab notebook

Overview

Assessments from the first cycle of LSST data indicate a need to further address image degradation. In particular, while optomechanical factors are being accounted for with high levels of precision, dome and mirror seeing remains uncharacterized (Urbach et al. 2026). More specifically, temperature fluctuations within the telescope enclosure have the potential to perturb the optical path, decreasing image quality in an effect known as dome seeing. Because the index of refraction of air is temperature-dependent, turbulent pockets of warmer and colder air alike cause refraction of incoming wavefronts. This translates to a broadening of the telescope's point spread function (PSF), the size of the image in the focal plane that captures an ideal point source, like a star.

BATPING will allow us to characterize temperature fluctuations within the telescope enclosure that perturb the optical path, decreasing image quality. BATPING offers the advantage that the speed of sound is much more sensitive to variations in temperature and hence refractive index. Additionally, BATPING's ultrasonic signals, even along the optical path, do not interfere with normal observations, nor do they rely on dome illumination.

My work documented on this page will track two subprojects: (I) construction of BATPING's audio hardware components, and (II) development of a machine learning (ML) approach to optimization of the acoustic signal.

Hardware Setup

The project apparatus can be characterized as an ultrasonic thermometer--measuring environmental temperature based on deviations in the speed of an ultrasonic pulse across a set path. Such a pulse is supplied by piezeoelectric ultrasonic transducers, organized into a hexagonal array to optimize amplitude and directionality (via near-field destructive interference); the pulse is reflected and then measured by an ultrasonic mic, such as the Dodotronic ultramic_384k_evo. Cross-correlation between the original pulse and the reflected pulse gives the time lag.

Project Log

Entry Date	Description of Work	Progress (failure modes, future improvements, assessment of workflow with Claude when applicable)
Week of 03/14/2026	• I was able to bring the sonic thermometer apparatus home with me for break, and I ran a couple of preliminary diagnostics on the hardware in preparation of reflection testing in the lab. I was able to confirm that the microphone can pick up a nicely powerful sine wave signal from the piezoelectric transducer array when it is powered by a waveform generator with a frequency of 40kHz and an amplitude of 1 V peak-to-peak.	• The Dodotronic ultramic (specs) has a 384 kHz sampling rate (with 190kHz bandwidth), so I attempted to configure the Audacity sampling rate accordingly.

	• The motion-sensing light systems found in some lab settings (potentially McKay 105...) emit a standing EM pulse of ~34kHz that has the potential to saturate the Dodotronic ultramic's reception of the piezo transudcer signal. Plots showing the power spectrum in a lab setting (with motion sensors) versus at my home (without motion sensors) are shown below.	Initially ran into issue with over-saturation of signal on 384kHz sampling rate, but this was due to the fact that the microphone has *manually adjusted* gain via a lever on the side of the mic. The issue was quickly corrected by setting the gain to switch 1 (-33dB). • When initially testing signal conservation in a lab environment, I found that the background ultrasonic pulse generated by motion-sensing light systems interfered with the microphone's ability to register the signal. However, when I tested at home in the absence of the structured noise, the microphone performed well in reading the directly transmitted signal *and* a reflected signal bounced off a right angle mirror. ☐ I'm wondering what a good metric is to measure the strength of the signal picked up by the mic relative to the original signal based on the information gathered in Audacity (which seems to record the waveform on a normalized unitless vertical axis).

April 19, 2026		
April 29, 2026		
May 1, 2026		
May 18-25, 2026	Worked through the first half of Indiana University's P400 Analog and Digital Electronics class (lab notebook here)	Very helpful exposure to DC/AC circuitry components (Op-Amps, Diodes, Transistors, ...) and hands-on exercises that stretched my brain quite a bit! I'm hoping to build on these skills during this summer during the hardware-focused component of BATPING research
May 26, 2026		
May 27, 2026		
May 30, 2026		
June 1, 2026	9am • confirmed that the mic receives both 1m transmitted and 2m reflection of low-frequency square pulse (2Hz, 10V_PP), to compare piezo response with result from stubbs goals for the rest of the day: ☐ test out some waveforms with a rise--40kHz pulse--fall structure; utilize (1) the rise and fall driving of the piezo array that we observed in the square pulse analysis and (2) its natural 40kHz resonance ☐ perform a characterization of the reflected signal strength as a function of larger distances (>1m) ☐ connect scope to output of agilent waveform generator (AWG) to see what Claude is really doing... ☐ run a second attempt at a matched filter method for measuring changes in traveltime, using manual adjustments of acoustic path length as a proxy for significant temperature fluctuation ☐ if bare bones matched filter works (update: it did) test out whether or not to use different preprocessing methods and specifications: **bandpass filtering** (35kHz-45kHz), **envelope extraction** (via the	**Reflection on Claude:** I was glad to have identified a couple reoccurring errors in Claude's workflow that have been dampening productivity.

	Hilbert transform), and **cross-correlation interval selection** (whole waveform (0-20ms), 5-15ms, 10-15ms, or 10-13ms)	
June 2, 2026	9am • wrote up logs and updates of work on Confluence • collected more data for matched filtering, but now with waveform given by the output of the bayesian optimization run on may 27 (as opposed to a simple gaussian envelope driven at 40kHz) • fixed square wave analysis from June 1 ◦ Note: The problem was not with Claude, but rather with the default export settings on Audacity. I had to manually indicate that I wanted to export only the selected mono track. The default option was to export the entire project, which compiled all of the tracks (adding to the raw data the single-period clipping that I cropped and put in a separate track for individual analysis), which explains the doubling feature. 6pm • run GCC-PhaT on matched filter with gaussian waveform • test different pre-processing methods (bandpass filter, envelope extraction)⟶both of these alterations to the raw data erode the phase information encoded in it. this ruins the phase transform that GCC-PhaT performs in frequency space because when the frequency information is normalized, regions of frequency space that have been artificially pruned (as are high/low frequencies by the bandpass filter) will be undefined when normalized (dividing by 0...) ◦ no pre-processing should be done when running the cross-correlation with phase transform	
June 3, 2026	8am • idea: reoptimize waveform using Bayesian optimization (Gaussian Process) or other ML method (CNN...) **but with the SNR of the cross-correlation function (CCF) as the objective function**. the rationale is that our end usage of the piezo response data is the xcorr with the template, so it would be wise to optimize with respect to the peak sharpness in the CCF, rather than the waveform peak, which is only an intermediary.	

June 5, 2026	7am goals for the day: ☐ perform matched filtering and analysis on optimized waveform recordings taken on June 2 ☐ Update: the piezo response when driven by the optimized input waveform is exceptionally low (~0.02); I am redoing the recordings and will locate the source of the signal weakness ☐ as suggested on June 3, re-run the Bayesian Optimization method, but now with the objective being the FWHM of the *cross-correlation plot* as opposed to the FWHM of the piezo response waveform. the end measurement which we are trying to make as precise as possible is the signal traveltime, which depends directly on the cross-correlation lag and only indirectly on the piezo response itself (though we would expect the two to be correlated)	
June 7, 2026	• pack up BATPING from Bloomington, relocate back to McKay 106 for the summer	
June 8, 2026	Goals for the day ☑ update confluence page through today ☐ Mostly done! but still need to write detailed summary of Bayesian Optimization work on **May 27**, as well as earlier dates when I was getting situated with everything (and trying out methods no longer being explored, like the gradient descent and simplex optimization methodologies) ☑ integrate new Waveform Generator into batping circuit. the Siglent Waveform Generator (SWG) is preferable because it uses standard SCPI command syntax and VISA-moderated interface with computer, very similar to Agilent WG, whereas I have been unable to upload successfully to the waveStation. ☑ Wrote the adaptions to the code for the SWG! I will test arbitrary waveform upload once I confirm more basic waveform results with the SWG. UPDATE (June 8, 6pm): This was an oversight. The Siglent Waveform Generator does not seem to use SCPI command syntax, even though it may use VISA for interface. Instead of a a set of commands defining remote control instantaneous arbitrary waveform upload, it uses EasyWaveX, a very low-tech GUI-based approach where the user can “draw” waveforms to upload. This WG will not be feasible, I have to go back to the waveStation. ☑ confirm that uploaded waveforms are being driven by WG as expected (using oscilloscope) ☐ See above ☑ set up piezo speaker, microphone in acoustic path for measurement in the lab ☑ characterize lab noise signature (will there be 33kHz from motion sensor for lights?) ☐ **Not 33kHz, but rather 25kHz...** ☐ re-record matched filter data for both Gaussian and optimized waveform to confirm repeatability of results obtained at home ☐ re-run Bayesian Optimization, but with the objective being minimizing the FWHM of the cross-correlation function (CCF) peak, rather than that of the piezo response waveform itself	
June 9, 2026	Summary of work for the day • switched back to Teledyne Lecroy waveStation 2052 WG	Reflection on Claude: Troubleshooting SCPI command syntax and communication issues

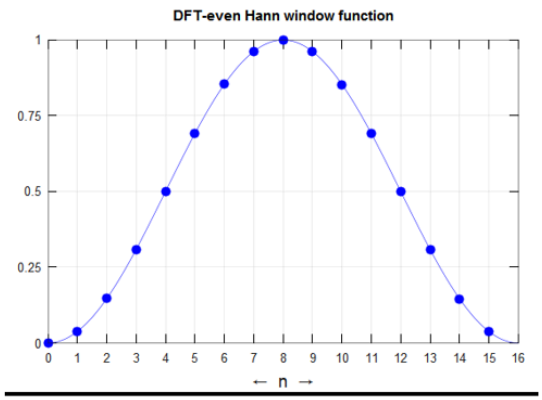
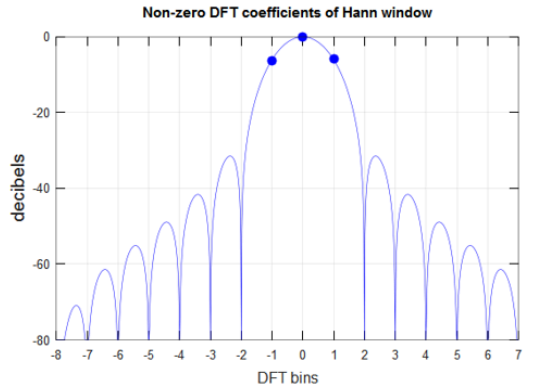
	• troubleshot arbitrary waveform code--the waveStation crashes when I attempt to write very simple waveforms using the WAVECSV command • varied modes of upload: **USBTMC** (USB Test and Measurement Class) is a standardized protocol that lets test instruments act like GPIB devices, while **USB RAW** is an unstandardized mode ◦ **USBTMC** is recommended in general, and I noticed (using the NI MAX and VISAIC applications from National Instruments) that the computer can only recognize the waveStation when it is set to **USBTMC** ◦ However, this demo from Teledyne Lecroy mentions switching to **USB RAW** for waveform upload specifically ◦ UPDATE: the waveform generator crashes (screen goes static) under **both** modes during upload... • Registered for Teledyne Lecroy account (1-2 business days for approval) and emailed a factory applications engineer about expedited access	with the WaveStation has been one of Claude's weak points. I find that Claude will pretty easily abandon the objective (of writing code that successfully automates the process of uploading a waveform) and quickly jump to suggesting alternatives (like using a USB stick, which is not feasible for the iterative usage I anticipate, like for Machine Learning optimization, for example). I think I definitely needed to rely more on planning mode in today's session, and this holds for troubleshooting in general. Otherwise, Claude is sort of aimless. I am also interested in making a 'skill' for Claude to apply when writing (and troubleshooting) command syntax for waveform upload.
June 10, 2026 (7)	Goals for the day: ☑ correct Cross-Correlation plots that had erroneous envelope extraction ☐ do inventory of circuit components in the lab; items list: Protoboard (like the PB-503), ... ☐ email electronics shop to request temporary usage of other parts if needed ☑ test two new methods of communication with the Siglent SDG ☑ (1) pure Python backend via PyVISA + PyVISA-py, and ☑ (2) raw TCP Sockets UPDATE: LAN connection using TCP/IP Socket worked!	

	☐ ~~once account registration is approved, test communication with the WaveStation after downloading the VICP Passport~~ UPDATE: i did get an account approved, but this is no longer needed!! ☐ finish BJT lecture for p400	
June 11, 2026 (8)	• Recorded rounds of Bayesian Optimization with new objective, namely FWHM of autocorrelation peak ◦ autocorrelation (as opposed to cross-correlation with some template) was chosen to limit the evaluation of waveforms to the characteristics that they possess, independent of any stored template or environmental conditions ◦ optimized_waveform_v2: N=200 iterations	
June 12, 2026 (8)	Motivating thought: optimization rounds v2 and v3 (with the new objective) produced 31% and 38% improvements in the AFC peak FWHM, respectively. These gains are significant, but the jump from 200 to 1000 iterations was only met with a 7% marginal improvement. I think that this is due to the limitations of the v1-3 parametrization: *The compensation functions are the only source of deviation from a strict gaussian.* In this way, the first three rounds of optimization have been very restricting, leaving simple sinusoidal bursts (at pre-set positions) as the only means of deviating from the gaussian structure. This leaves an open question of how to parametrize the waveform to yield a more expansive optimization. Goals for the day: Backlog ☐ finish BJT lecture for p400 ☐ do inventory of circuit components in the lab; items list: Protoboard (like the PB-503), ... ☐ go to electronics shop to request temporary usage of other parts if needed→go Monday June 14!! New work, looking towards pseudo-random noise (PN) sequences ☑ Characterize piezo transfer function — measure frequency response to know actual usable bandwidth→UPDATE: took measurements on June 12, processed + analyzed data on June 13-14, see here ☐ Implement Barker-13 code, with BPSK (Binary Phase Shift Keying) as baseline coded waveform — Generate 13-chip sequence — BPSK modulate onto 40 kHz carrier with Gaussian chip pulse shape	

	— Upload to SDG, test correlation peak width, compare with theoretically 'perfect' BC' peak	
June 13-14	Goals for the weekend: ☑ Process and analyze piezo characterization data (chirp test) from Friday → Piezo Bandwidth Characterization ☐ Write up my own (NOT claude's) current understanding of the Binary Pseudo-Random Sequence (BPRS) method and its derivations in digital circuit logic and signal encoding/processing Backlog: ☐ Implement Barker-13 code, with BPSK (Binary Phase Shift Keying) as baseline coded waveform — Generate 13-chip sequence — BPSK modulate onto 40 kHz carrier with Gaussian chip pulse shape — Upload to SDG, test correlation peak width, compare with theoretically 'perfect' BC' peak ☐ Make circuit component list for electronics course	
June 15	Recap • Began implementing Barker-7 BPSK waveform for SDG1062X; reviewed BPSK modulation scheme and SDG arbitrary waveform upload protocol in preparation for coded waveform testing	
June 16	Recap • V1: initial Barker-7 BPSK attempt failed — 40,000-sample waveform exceeded SDG's 16,384-point limit; carrier compressed to ~400 Hz due to sample rate mismatch • V2: fixed by constraining sample count and computing rep_rate = SDG_SAMPLE_RATE / len(waveform) to preserve 41.2 kHz carrier. New problem: no inter-pulse gap → piezo driven continuously → blended correlation peaks; TOF wildly inconsistent (1940–9229 µs), FWHM stuck at 10.4 µs (code was measuring carrier oscillations, not the correlation envelope)	
June 17	Continuing the Barker Pipeline improvement • V3: added inter-pulse dead time by running at 40 Hz rep rate (~21 ms dead time per period); waveform embedded at start of 10,000-sample buffer with trailing zeros • V4: key shift in analysis — dropped cross-correlation with template in favor of autocorrelation of the received signal itself. Motivation: template mismatch (theoretical ≠ what piezo actually emits) was producing arbitrary TOF values; autocorrelation	

	is self-referencing and naturally encodes both FWHM and TOF: main peak at lag=0 gives resolution, secondary peak at lag=TOF gives time of flight. Also added Hilbert envelope extraction to measure correlation envelope width rather than carrier oscillation period	
June 18	Continuing the Barker Pipeline improvement • V5: fixed two bugs from V4 ◦ (1) sidelobe level now reported relative to main peak (Barker-7 theoretical = 1/7 → -16.9 dB, not absolute dB), ◦ (2) transfer efficiency now normalized by signal length (earlier version was comparing peaks of different-length arrays, meaningless). Added full data archival: WAV files + manifest.json per run • Full 9-point sweep: cycles/chip ∈ {16, 20, 24, 28, 32, 36, 40, 44, 48}. FWHM scales linearly (495 → 1292 µs), consistent with FWHM ≈ T_c = N_cyc/f_carrier. Measured sidelobes -12 to -14 dB vs -16.9 dB theoretical. TOF: 5716–5734 µs (consistent). ◦ Transfer efficiency improved -45 to -36 dB UPDATE: later identified as artifact (efficiency = len(wf)/len(recording), no acoustic information)	
June 19	Exhaustive recording session before anticipated absence from lab • Full pulse compression survey at ~1 m: compared baseline Gaussian, optimized V3, all Barker lengths (2, 3, 4, 5, 7, 11, 13 chips at 24 cyc/chip), LFM chirp (38–44 kHz, 2 ms), MLS sequences (periods 7, 15, 31), Golay pair A and B • Multi-distance TOF test with Gaussian, optimized V3, Barker-7 (24 cyc) at 7 distances from 37 to 225 cm; Barker-7 TOF detection failing at ≥147 cm (peak finder returning 500 µs fallback)	
June 20-29		
June 30-July 1	• 25 kHz noise investigation: power spectrum analysis of archived WAV files revealed environmental interference spike. Baseline: ◦ optimized_v2_response.wav shows -26 dB at 25 kHz,	

	◦ optimized_v3_response.wav shows -20 dB at 25 kHz, but ◦ barker7_24cyc_received.wav (June 18) shows only -115 dB • This retroactively explains the oscillatory autocorrelation envelope seen in V3 during earlier analysis: elevated 25 kHz noise during the June 11 session was contaminating the autocorrelation, not a property of V3 itself. A 30kHz high pass filter should always be implemented ◦ UPDATE: Prof Stubbs suggested a “notch” filter around 25kHz--this should work even better	
July 2	Goals for the day • Extend Barker-7 sweep downward in cycles/chip to empirically locate the FWHM saturation floor (Claude predicted theoretically at ~6 cycles, where $T_c \approx \tau_{piezo} \approx Q/(\pi f_0) \approx 154$ µs) ◦ Ran lowcyc sweep including 2, 4, 6, 8, 9, 10, 11, 12, 14, 16 cyc/chip in addition to previous 16–48 range • Start LFM chirp sweep: ◦ tested bandwidths from 1.7 kHz (within -3 dB BW) to 12 kHz, duration sweep at 3.75 kHz (-6 dB BW), and amplitude-compensated variants using $1/\lvert H(f)\rvert$ pre-distortion from stepped-sine characterization. ◦ Best result: 8 kHz compensated, 10 ms → FWHM = 169 µs, new best, beats Bayesian-optimized Gaussian V3 (260 µs). FWHM ≈ 0.886/B confirmed	
July 3-5	Work from the past three days: • Extended chirp sweep to much wider bandwidths (up to 30 kHz): FWHM saturates at ~88–99 µs for BW ≥ 16 kHz — this is a noise/multipath floor, not a piezo bandwidth limit. Pushing further doesn't improve FWHM • Applied Hann window to autocorrelation lag axis, this worked horribly ◦ Before, I was using Tukey windowing, which essentially provided a mild sine buildup in amplitude at the beginning and end of the recording duration. I thought that maybe the Tukey windowing was not smoothing the edges	

	of the recording, which would could cause distortion. Hann windowing, on the other hand, is a raised sine envelope applied throughout the waveform recording, which I thought could work better: ◦   ◦ Hann windowing roughly halved measured FWHM across configs--switching back to Tukey • Best practical configuration identified: hf_m15dB asymmetric 2 ms compensated (rect window), FWHM = 174 µs, σ_TOF = 0.272 µs → ~0.026°C per ping without any averaging • Found that over-compensation (sweeping to -20 dB piezo rolloff points) creates W-shaped transmitted envelope, which generates spurious cross-terms in the autocorrelation at ~1.75–3 ms lag (not affecting TOF peak at 5.7 ms, but complicating interpretation). Practical compensation limit is -12 to -15 dB	
July 6-7 (8, 7)	Recap • Ran Bayesian optimization for LFM (Wiener filter approach) and NLFM waveforms; evaluated all candidates through Stubbs's 9-second acquisition pipeline • Stubbs pipeline results as of now: NLFM best at σ_TOF ~ 0.287°C/ping; flat chirp and lightly	

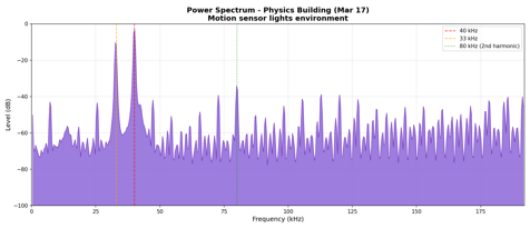
	compensated chirp (comp-light) performed comparably ◦ NOTE: optimizer-reported σ_TOF and Stubbs pipeline temperature precision are not directly comparable — optimizer runs short recordings and reports per-period ACF precision, Stubbs pipeline captures a longer multi-condition sequence. This explains why the best reported σ_TOF value is much larger than what we've previously gotten • Hardware updates: ◦ Siglent SPA1010 power amplifier in hand. Let's see if we can use it: ▪ We're currently at currently at -6 to -8.5 dBFS with 2 Vpp. However, given that the UltraMic is at switch 1 (-33 dB), the max safe drive is ~5 Vpp. Therefore, we can't really use the Siglent amplifier in a meaningful way until we switch to the better analog mic...	
July 8 (6)		
July 9 (9.5)		
July 13 (7)	Goals for the day: ☐ Document, document, document. Write up the most recent updates to the double-template matched filter pipeline, in particular the new full-CCF Least Squares fit developed on Saturday. ☐ Troubleshoot the current issues with matched filter analysis of the 1hr recording from July 10, and write up the issues still persisting at the end of the day ☐ Map out the transfer function of the new microphone + piezo array system ☐ If time provides, re-record the alpha/chirp sweep with the old UltraMic (for repeatability) *and* with the new ___ mic (for enhanced sampling rate)	

L-Path Signal Reading and Structured Environmental Noise

To first confirm that the transducer apparatus was working, I tested (under two different environments) whether the microphone could pick up a transmitted signal when placed: (1) directly in front of the piezo array (path length: 1m) and

(2) aligned with a makeshift 90° reflecting mirror (path length: 2m).

Measurements in the physics building were hindered by the presence of ultrasonic motion sensors:



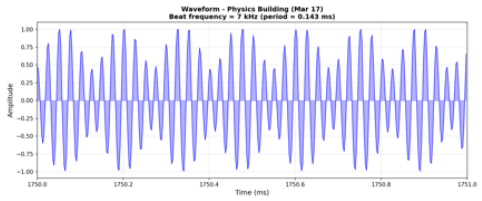


Figure 2: waveform of total received signal displaying ~7kHz beat frequency

A less noisy environment allowed for clean measurement of direct, and, as shown below, even reflected signals:

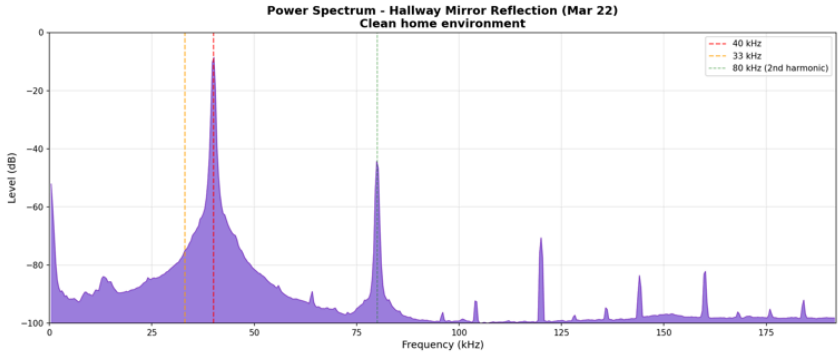
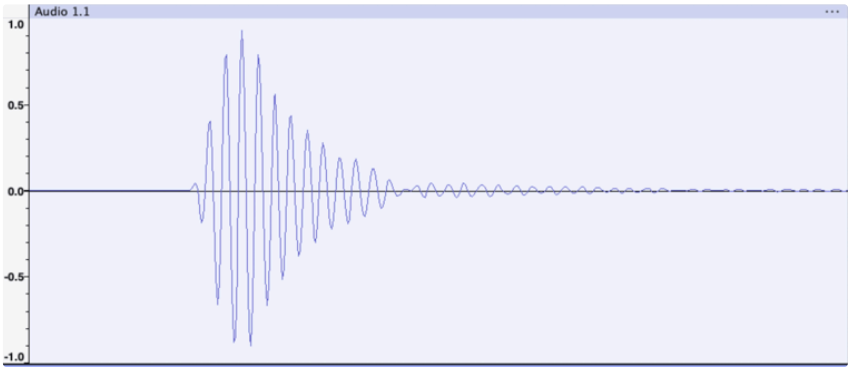


Figure 3: power spectrum recovered from L-shaped reflected path in quiet environment

Note: the vertical axis of the power spectra plots (“Level”) is measured in deciBels relative to Full Scale (dBFS), where 0 is the maximum possible amplitude the mic can pick up before clipping.

Low-Frequency Square Wave Pulse

Sending 2Hz 10V peak to peak square wave works really well. Here the mic is positioned 1m away from the piezo speaker to receive its transmitted signal head-on

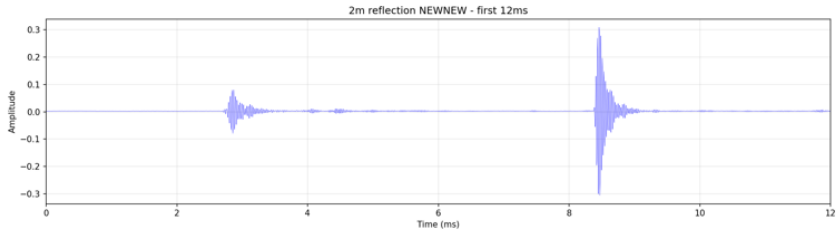


transmitted waveform, measured 1m away from piezo array speaker

It showcases ringdown very similar to what Professor Stubbs observed from his square wave tests.

Now, let’s measure the reflected waveform, positioning the dodotronic just next to the piezo speaker, both of which 1m away from a flat glass surface reflector (replace with corner cube when back in lab!!!). After many trials and tribulations

with displaying the waveform (see note in June 2nd report, I messed up audacity export), I'm able to cleanly see the mic picking up the signal (1) right as it is outputted by the array, and (2) as it's returning from its 2m roundtrip travel.



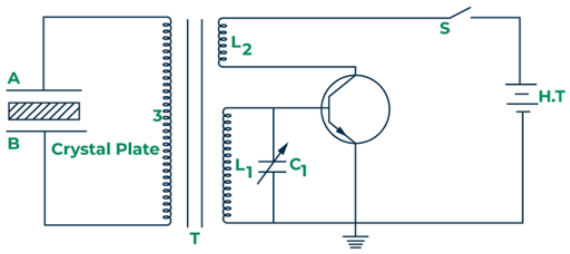
outgoing (~8cm) and reflected (2m round trip) signals, as picked up by the mic stationed next to the piezo array in the reflection configuration

The huge increase in reflection signal amplitude (relative to outgoing) is due to the limited omnidirectionality of the mic.

Theory

Piezoelectric transducers convert **changes** in voltage to mechanical displacement. Why? Well the initial signal (driven by the AWG) is a square wave voltage. When the current carrying this signal arrives at the metal electrodes (A and B) surrounding the piezo crystal, they accumulate positive and negative charge respectively (in the same manner as a capacitor), which results in the changing of the atomic spacing within the lattice of the piezo crystal. The best way I can think about this is the following: applying an electric field to many-electron systems will change the minimum potential energy (and the ground states) of the electrons in the crystal. This polarization will change the positions of the atoms, sending a pressure wave originating at the transducer surface, propagating outward.

Even though we're currently only using the piezo transducers in the speaker direction, they can also be mics! If the initial signal is an acoustic square wave (oscillation of air pressure through space) then it exerts an oscillating force on the transducer surface when it changes the pressure there. This causes the piezo crystal to alternate between compression and expansion. The crystal sits between two metal electrodes (A and B), which accumulate positive and negative charge respectively via the shifting of atoms towards either electrode face (is this similar to the mobile charges in a p-n junction diode??). Thus, differences in potential accumulate and an AC voltage signal propagates with a matching frequency. So, without changes in pressure, there is no mechanical vibration of the piezo crystal.



General piezoelectric transducer circuit diagram

Chain of interactions:

electrical signal ⟶ E-field exerted between electrode faces ⟶ atomic displacement and crystal deformation (i.e. Strain) ⟶ mechanical signal

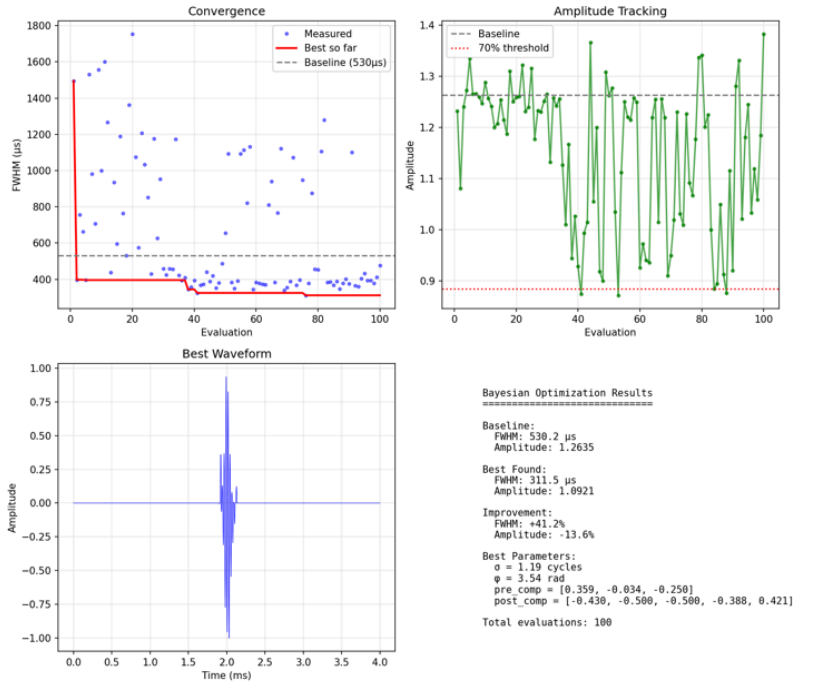
air pressure oscillation ⟶ mechanical vibration of the transducer face ⟶strain in piezo crystal ⟶ electrical signal

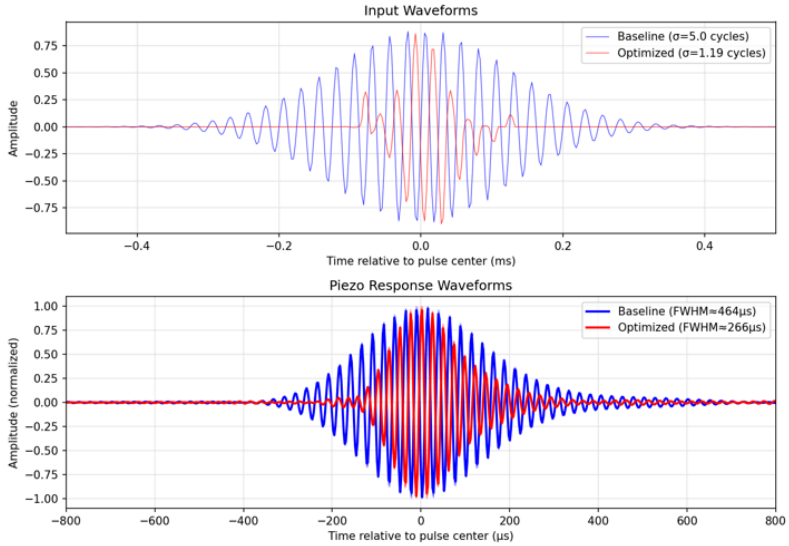
Input Signal Optimization

Bayesian Optimization v1

With the objective function minimizing the FWHM of the piezo impulse response waveform:

Mosaic macros cannot be exported to PDF.

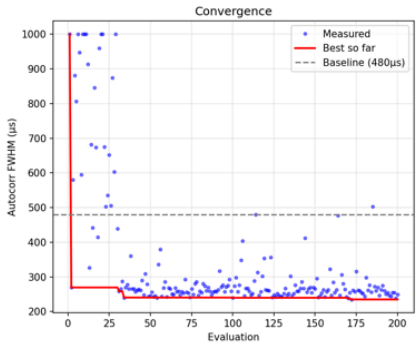


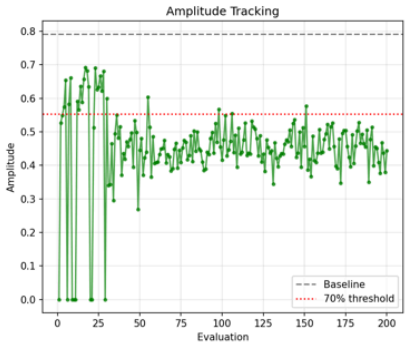


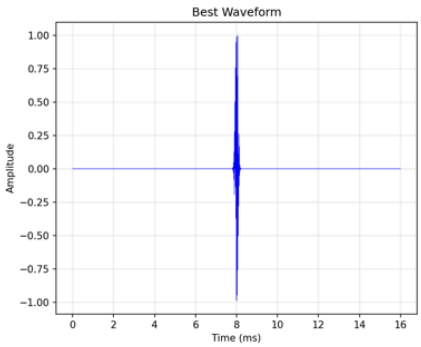
Bayesian Optimization v2 (June 11)

With the objective function minimizing the FWHM of the Auto-Correlation Function (ACF) of the piezo response waveform with itself:

Mosaic macros cannot be exported to PDF.








```
Autocorr FWHM Optimization Results (SDG1062X)
================================================

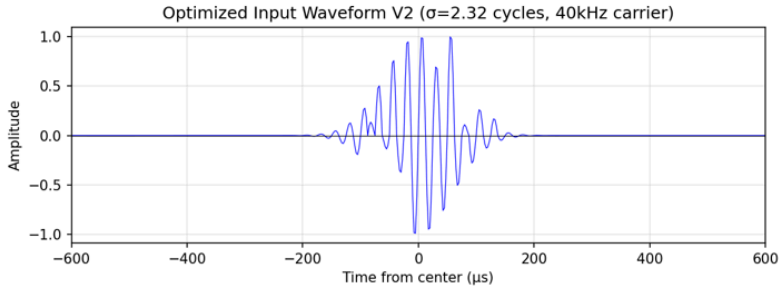
Baseline:
  Autocorr FWHM: 479.5 µs
  Amplitude: 0.7915

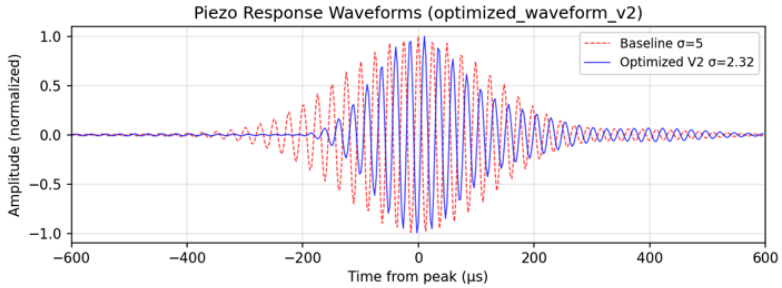
Best Found:
  Autocorr FWHM: 293.1 µs
  Amplitude: 0.5669

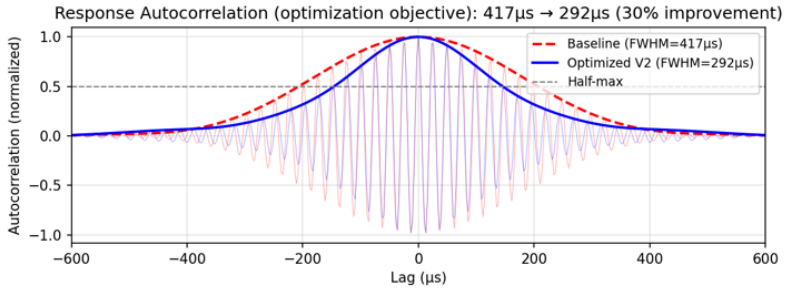
Improvement: +38.9%

Best Parameters:
  σ = 2.32 cycles
  φ = 0.00 rad
  pre = [0.50, 0.50, 0.50]
  post = [-0.17, 0.50, -0.50, 0.17, 0.30]

Total evaluations: 200
```

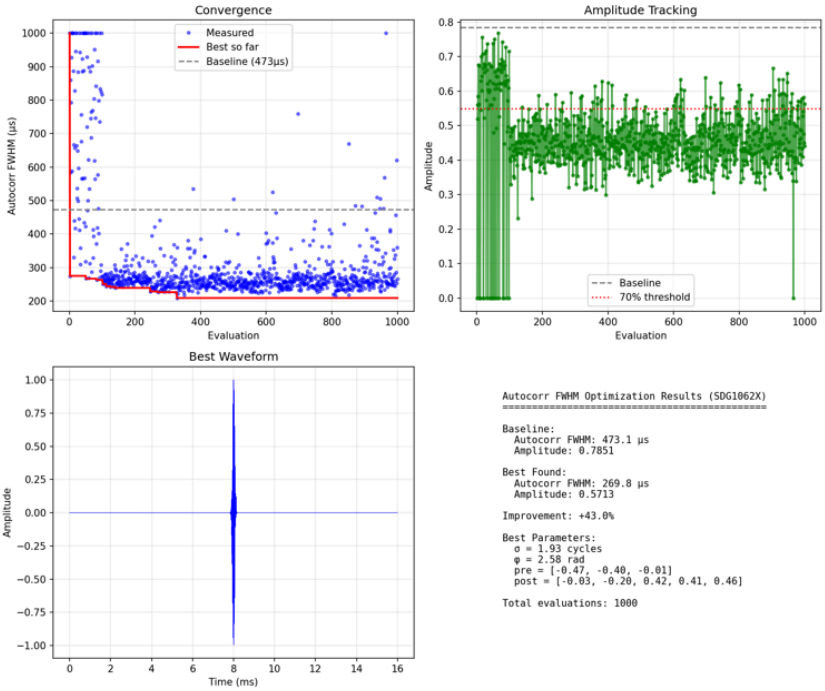


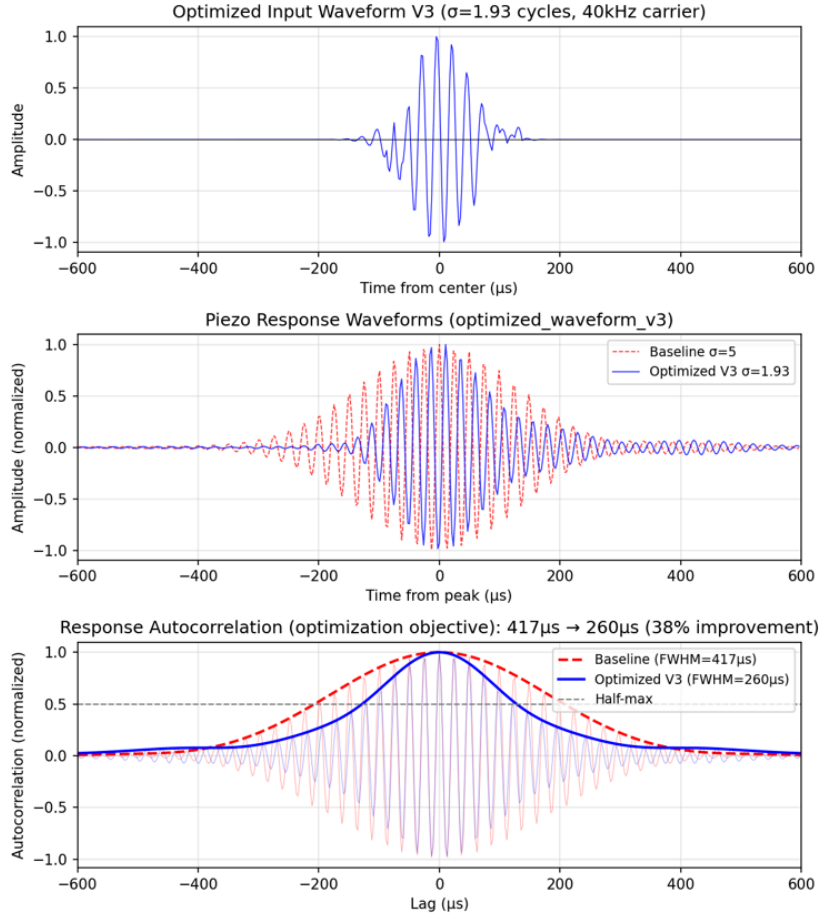




Bayesian Optimization v3 (June 11)

With the same objective function as the v2 optimization, but now with N=1000 iterations:





Matched Filtering

One formatting of the goal of BATPING is to measure the line integral of temperature fluctuations along potential stellar optical paths. One method of accomplishing this goal is to perform matched filtering, which involves the construction of

a **template waveform** (usually taken as the average of many piezo responses recorded in a short time interval). I then run a cross-correlation of this template with a signal recorded after some sort of perturbation to the system. Nominally, this perturbation would be a fluctuation in the temperature--if the matched filter is constructed in standard conditions, then the signal's acoustic traveltime will differ measurably when the signal encounters fluctuations in the temperature, or when the temperature changes in the time between the template and signal recordings. **Without thermodynamic insulation of the hardware and acoustic path or precise environmental temperature control, I am simulating temperature fluctuations using a different type of physical perturbation to the signal, namely manual changes to the path length.** Moving the reflector x cm closer to the piezo array + mic is equivalent to a temperature change of y degrees Celsius.

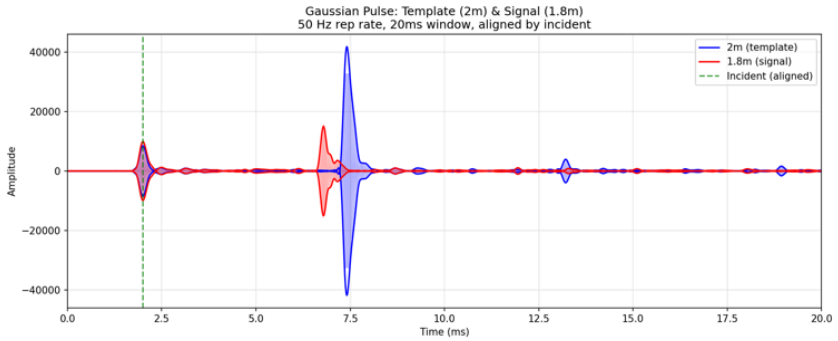
Physical Setup and Recording

The piezo array and microphone are positioned 1m away from a reflector (a flat glass surface for now, but will be replaced by a proper corner-cube reflector). Then, the Agilent Waveform Generator (AWG) drives an input waveform (initially a 10 V_PP, 2-cycle gaussian envelope with a 40kHz driving frequency), at a pulse frequency of 50Hz (one waveform sent every 20ms). Each 20ms time window contains two dominant features (1) an initial peak corresponding to the mic pickup of the outgoing signal (this is a transverse pickup, so the strength of the signal is low), and (2) a second peak corresponding to the mic pickup of the reflection after having traveled 2m round trip. After 10 periods, there is enough data to construct a template with N=10. The 20ms time windows are averaged, and the template is stored as a .npz file.

Then, the reflector is moved 10cm closer to the piezo speaker and microphone, such that the round trip traveltime is now 1.8m. The same input waveform is driven by the AWG, and a 20ms 2-peak segment of data is collected. The signal is stored as a .npz file.

Results: Using a Gaussian Pulse with a 40kHz Carrier (June 1, 2)

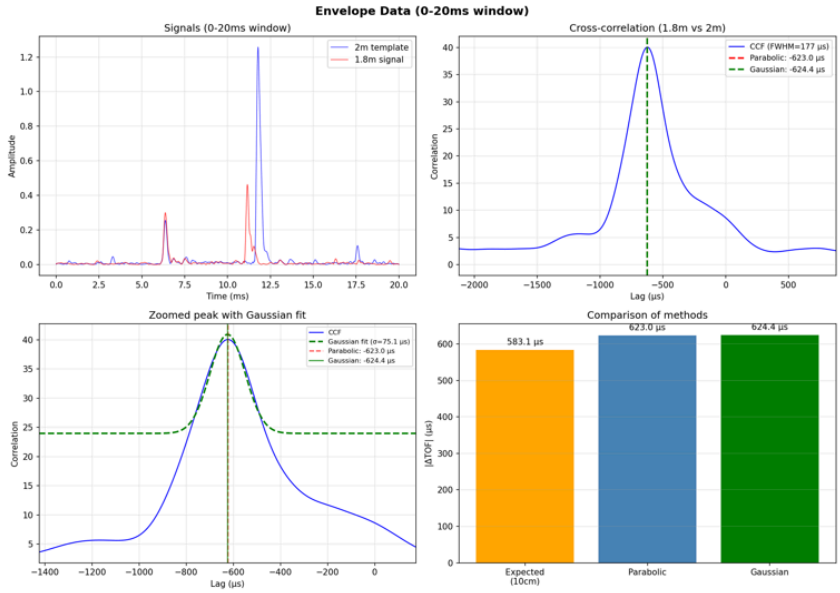
After taking data on June 1 (in the manner described above), I obtained the following template and signal waveforms when driving the piezoelectric array with a 10 V_PP two-cycle gaussian pulse (with 40kHz carrier frequency and 50Hz pulse frequency):



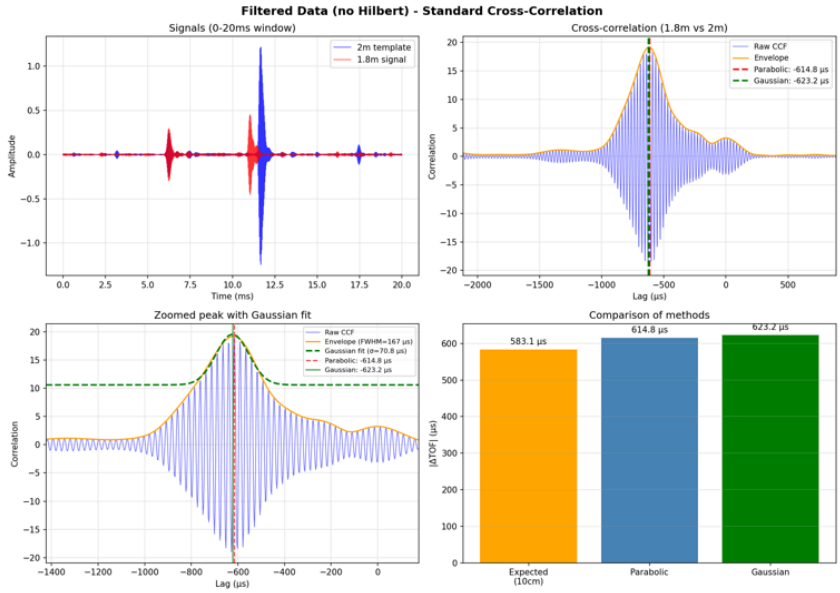
I'm not entirely sure why the amplitude of the 1.8m (signal) reflection is significantly lower than the 2m (template) reflection. This is something to explore further, but I anticipate it was most likely due to some sort of change in the environment between when the two waveforms were recorded (there could have been background noise during signal recording). It shouldn't affect the cross-correlation significantly...

With this data in hand, I tested out the effect of processing the piezoelectric response waveform in different ways on the precision of the matched filter, as determined by the width of the peak in the cross-correlation plot (CCP). I expected that minimizing the noise in the template and signal waveforms would sharpen the peaks, and thus lead to a narrowing of the CCP peak. In turn, I thought that bandpass filtering to the interval (35kHz, 45kHz) and extracting the

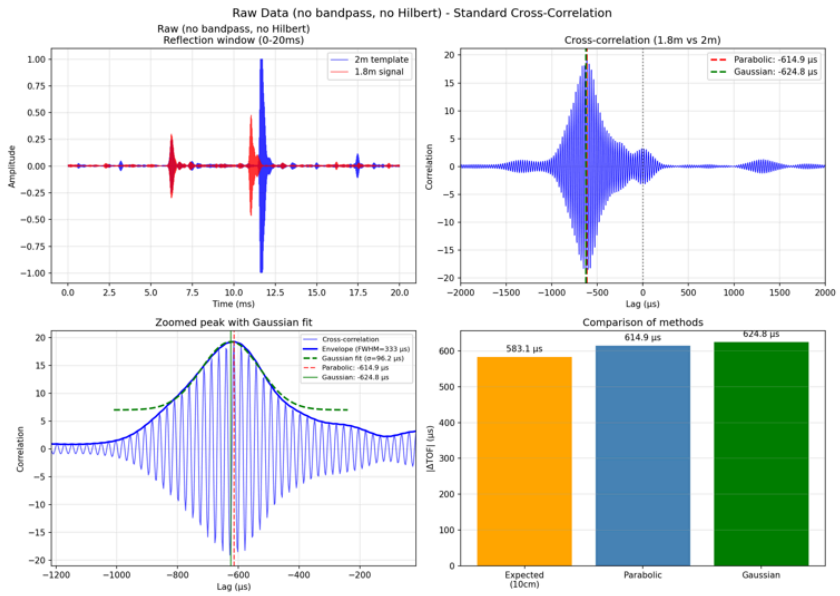
envelope of the waveforms via the [Hilbert Transform] would clean up the waveforms. I also tested limiting the time interval of the matched filtering to **only include the reflected peaks**, so as to eliminate side lobes corresponding to incident-reflection alignment. The following plots test these three features, namely **bandpass filtering**, **envelope extraction**, and **time intervaling**:



Results from performing **bandpass filter** and **envelope extraction** on full 20ms data

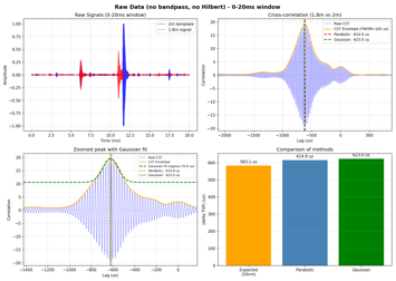


Results from performing **bandpass filter** but **not extracting the waveform envelope**, on the full 20ms time series

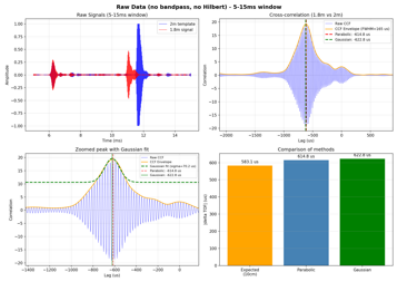


Results from running the cross-correlation on the full raw 20ms time series data (i.e. **no bandpass filtering nor envelope extraction**)

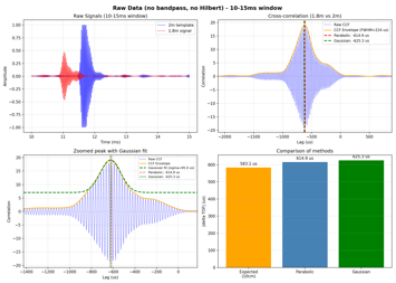
Now, the following four plots show the effects of choosing different **time intervals** to run the cross-correlation on, with ranges 0-20ms (the entire time series, including both incident and reflection peaks of both template and signal), 5-15ms (incident and reflection peaks still present, but noise before and after is cut out), 10-15ms (only reflection peaks present), and 10-13ms (very refined interval around reflection peaks)



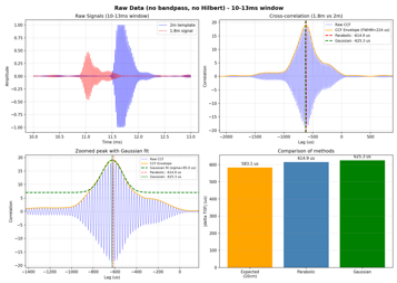
Cross-Correlation applied to time interval 0-20ms



Cross-Correlation applied to time interval 5-15ms



Cross-Correlation applied to time interval 10-15ms



Cross-Correlation applied to time interval 10-13ms

The results were actually in direct contrast to my expectation, but now make perfect sense. The cross-correlation is not necessarily optimized by sharp peaks alone, but can also be improved by distinct features in the waveform that cause a large drop-off in alignment over a small scale of lag change (i.e. $\Delta f_{CCF}/\Delta\tau$ is high). This explains the trend observed in each of the three preprocessing methods:

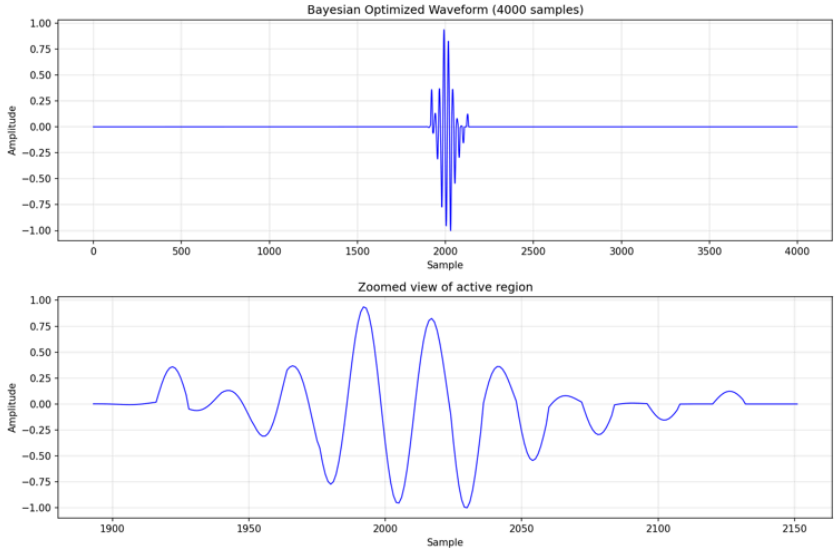
1. Envelope extraction is detrimental to the narrowness of the peak in the Cross-Correlation Function (CCF). This is due to the fact that there is useful phase information stored in the carrier oscillation (or any small-scale fluctuation in the waveform that occurs under the envelope).

2. Bandpass filtering has a similar effect for a similar reason. Now, the information loss occurs in frequency space--off-resonance frequencies can be very useful for the cross-correlation operation

3. Time intervaling is another example of information loss (and the same reasoning above applies)

Results: Using optimized_waveform_v1 (June 2, 5)

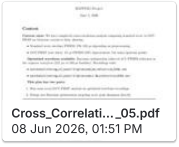
On June 2, I recorded template and signal data using the waveform produced by the Bayesian Optimization performed on May 27:



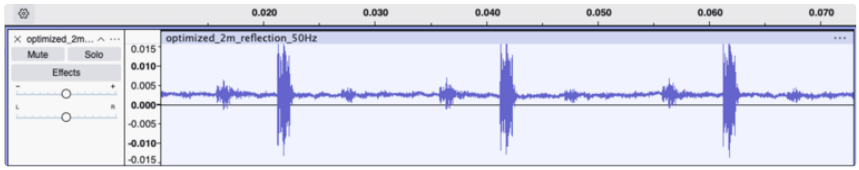
I prompted Claude to drive the optimized waveform we had on file with a pulse frequency of 50Hz. Recordings were saved as optimized_2m_reflection_50Hz_june2.wav, optimized_1.8m_reflection_50Hz_june2.wav. Today (June 5) I am finally getting around to constructing the matched filter template and performing the cross-correlation:

> Claude Prompt (in planning mode): let's get back to our goal of running the cross correlation for matched filtering (both standard xcorr, varying whether or not to do envelope extraction via hilbert and/or bandpass filter, AND gcc-phat, varying whether or not to do bandpass filter ALONE), *but now using the optimized waveform from the bayesian optimization run. this should be relatively straightforward given that we've already generated the code for it. after this is done, i also want to run a new optimization, this time going straight for the thing that we actually want to optimize, namely the sharpness of the cross correlation plot peak. while one would definitely expect the sharpness of the piezo response waveform to determine the sharpness of the CCP peak, there could be some merit to optimizing the end goal entity. we should talk about this more

Output:

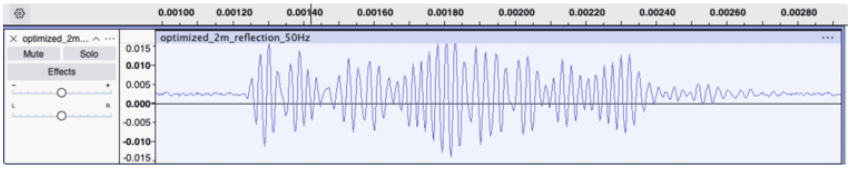

Cross_Correlati..._05.pdf
08 Jun 2026, 01:51 PM

This didn't go as expected--Claude generated a reasonable two-part plan (above) but was unable to identify appreciable signal peaks in the .wav files. Looking at them myself, the piezo response waveforms I do find are extremely small in amplitude and look nothing like the reported piezo response for this optimized input waveform:



Multiple periods of piezo response (incident waveforms are the smaller-amplitude packets, reflected waveforms are the larger-amplitude packets)

The amplitudes here are remarkably low--the larger-amplitude waveform achieves ~0.015 at best. Additionally, this piezo response is pretty horrible, certainly not what Claude had reported the optimized waveform as achieving previously. Let's zoom in on the piezo array's response to one of the reflected signals:



Single piezo response waveform (one of the larger-amplitude reflected waveforms)

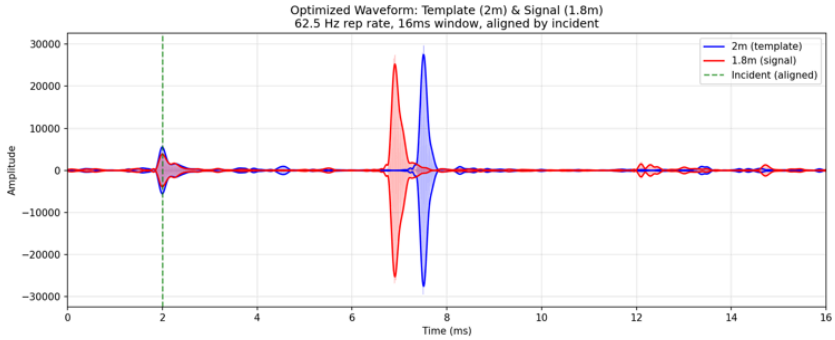
The plot of the optimized waveform in the Bayesian Optimization section shows that it is pretty unambiguously 40kHz, but this behavior looks nothing like the piezo ringing at resonance, so something is wrong in the upload process. I think the piezo is being driven very far off resonance, because even if it wasn't an optimized input waveform, if it's 40kHz it should still ring nicely. Let me recheck the optimized waveform npz file for data format, length, etc.

Here's the problem: **the optimization was conducted with pulses sent at a frequency of 250Hz (this is even documented above). However, I have been working the matched filter with pulse frequency of 50Hz.** This led to Claude taking the optimized waveform, which resides in a 4ms time window, and **stretching the waveform to fit in a 20ms window**, leading to a decrease in frequency by a factor of 5 (40kHz⟶8kHz).

So, to use the optimized waveform as it is saved, simply pad the end of the data array with enough 0's to fit whatever period length we want that is larger than 4ms. Via this method, a period of 20ms would require padding until the time series is 20,000 samples. But the Agilent WG has a 16,384 point limit for arbitrary waveform upload.

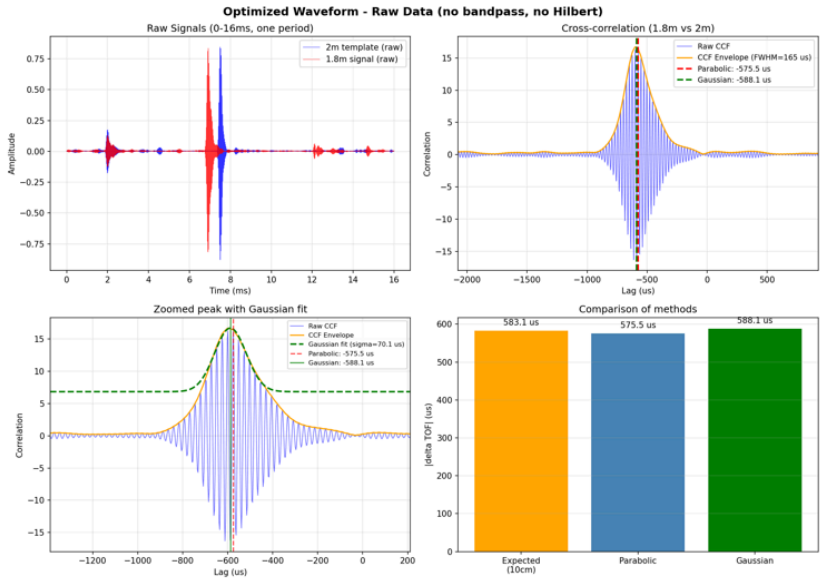
Instead, I will use a pulse frequency of 62.5Hz, which corresponds to time series of 16,000 points, within the AWG limit.

The plots of the signal and template are below:



Template waveform (average of N=10 piezoelectric speaker responses at 2m path length) given in blue, signal waveform (piezo speaker response at 1.8m path length) given in red

Running the cross-correlation, using no bandpass filter nor envelope extraction (as informed by our work on the Gaussian pulse), yields the following



GCC-PhaT (Generalized Cross-Correlation with Phase Transform)

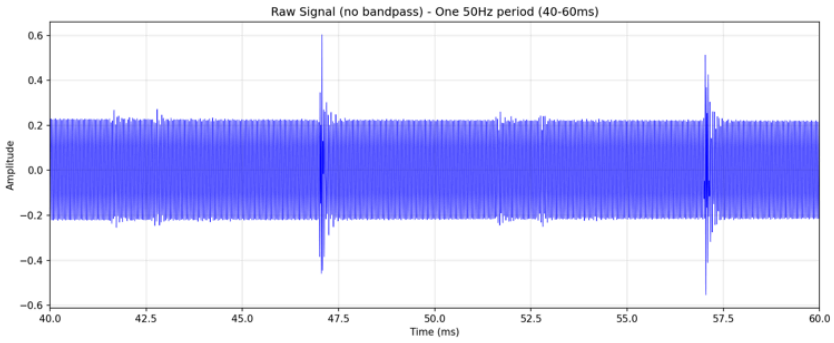
The following was adapted from my phys15c final project report

Mosaic macros cannot be exported to PDF.

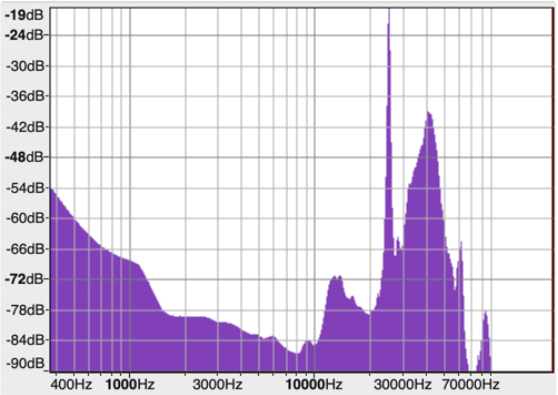
The effectiveness of the Phase Transform (normalizing in Fourier space) stems from discarding amplitude information in favor of phase difference. PhaT is normally an enhancement to the standard cross-correlation (xcorr) in contexts where multipath echoes and reflections are prominent or where SNR is low (i.e. noise is high). These conditions don't immediately strike me as describing my current BATPING setup, so I am testing whether GCC-PhaT offers an improvement to the traveltime change calculation.

In-lab Replication of At-home Results (June 8)

Let's start with a simple 50Hz square wave to observe the twice-per-period driving of the piezo transducers by the rise and fall of the square wave.



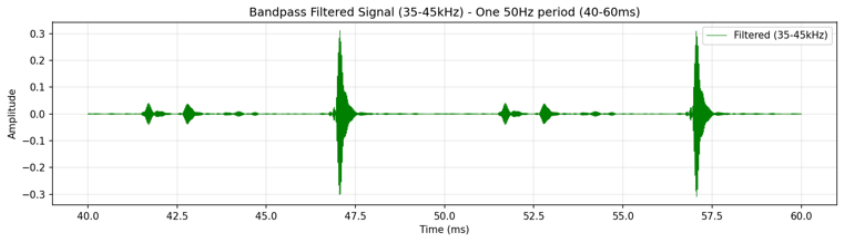
The mic pickup is very noisy without a bandpass filter. What is the power spectrum like?



Power spectrum of
50Hz_squarewave_10V_PP_inlab_jun8_TRIAL2.aiff

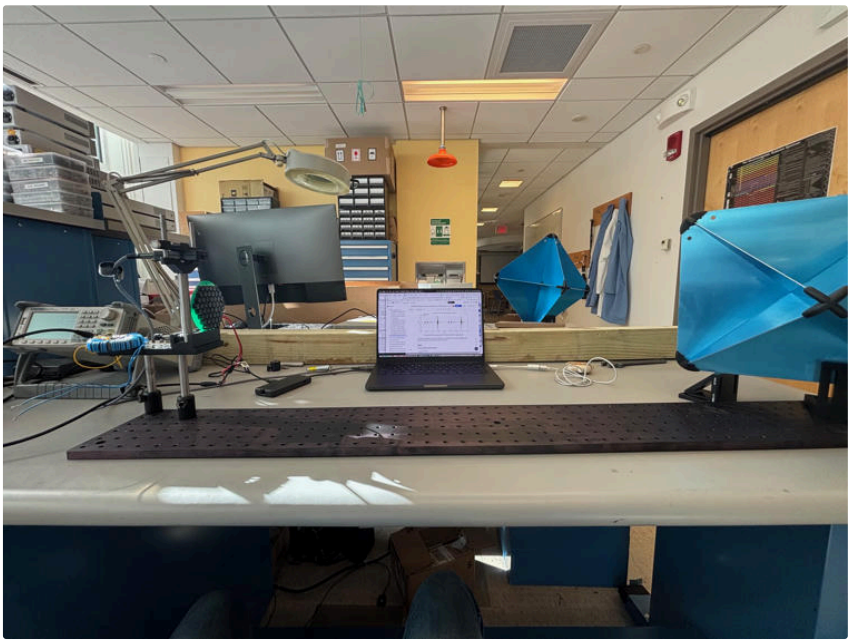
There is a peak at 24.98kHz (of ~-19.8dB) that is completely overtaking the signal—>there is some sort of ~25kHz noise in the lab. It could be from the motion sensors in the lights, or from the laser room, or any other sort of active piece of technology.

The cleaned up version (bandpass filter 35kHz-45kHz) looks better, though it reveals a mysterious small-amplitude peak (definitely multipath), presumably 1ms after the microphone picks up the incident signal 10cm away from the speaker:



First Period of 50Hz_squarewave_10V_PP_inlab_jun8.aiff after bandpass filter

That second small peak may be due to reflection off of the table, or some sort of multipath in the packed environment of the lab. Here is the current setup:



Debugging (June 9-11)

The first major roadblock that I encountered was with the Siglent SDG1062X waveform generator, which (to my knowledge) lacks SCPI support for arbitrary waveform upload—the printed user manual that came in the box states that this functionality requires the EasyWaveX GUI software, which I don't think can be programmed for automatic waveform uploaded (it requires users to "draw" their waveforms).

I pivoted back to the Teledyne LeCroy WaveStation 2052, which has a documented SCPI command (WVCSV) for uploading arbitrary waveforms via USB. The communication stack works as follows:

1. SCPI (Standard Commands for Programmable Instruments) provides the text-based command syntax (e.g., *IDN?, BSWV, WVCSV M50,WAVENM,TEST,CSVLENG,8,CSVDATA,Amplitude,0,0.5,1,...) that are **human-readable**.
2. VISA (Virtual Instrument Software Architecture) is the underlying I/O layer that handles the actual USB communication. I have NI-VISA installed, and basic SCPI commands (*IDN?, BSWV, ARWV) work correctly—I can query the device, set waveform parameters, and select built-in arbitrary waveforms.

However, when I send the WVCSV command to upload custom waveform data, the device immediately crashes: the screen goes static and requires a power cycle. This happens regardless of command variations (short form WVCSV vs. long form WAVECSV), data content (floats vs. integers vs. all zeros), or point count (8 or 32 points). A factory reset did not resolve the issue.

The SCPI manual states that USB communication "utilizes Teledyne LeCroy's VICP protocol" and requires installation of the VICP Passport—a proprietary plug-in that extends NI-VISA to handle LeCroy's custom protocol framing. Yep, just confirmed with NI MAX and NI VISAIC—WaveStation only shows up for I/O as USBTMC. But if you keep it on USBTMC when you try to write, it crashes. Conversely, if you try to change it to USBRAW to write, it doesn't even show up to the computer—likely because USBRAW requires VICP Passport to provide the driver layer.

Downloading VICP Passport requires a Teledyne LeCroy account, and my registration is pending approval (as per the website, they are "reviewing my application and [I] can expect to hear from [them] in 1-2 business days"). I called the Teledyne LeCroy customer support line (no response, but I left a message) and emailed their customer support address.

UPDATE (June 10): I have now transferred back to the Siglent SDG1062X, and, at the advice of Prof. Stubbs (and Claude), I am using LAN connection now for Mac to SDG.

Setup:

Connect SDG ethernet → ethernet cable → ethernet-to-USB-C adapter → Macbook

System Settings → Network → Click on the connected device at the top (mine shows up as "AX88179A", which is the chip identification for the particular USB-C port I am using) → "Details" → TCP/IP

Then,

- Configure IPv4: Manually
- IP Address: 10.11.13.1
- Subnet Mask: 255.0.0.0
- Router: (leave blank)

(Note that these numbers are determined entirely from the settings on the SDG. To find its IP info, go to Utility → Interface → Lan Setup)

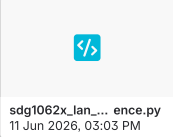
To confirm the connection, run

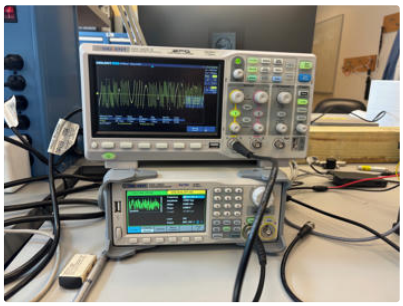
```
ping [SDG_IP_ADDRESS]
```

If successful, then you should see something like

```
64 bytes from [SDG_IP_ADDRESS]: icmp_seq=545 ttl=64 time=0.651 ms
```

Run this script:



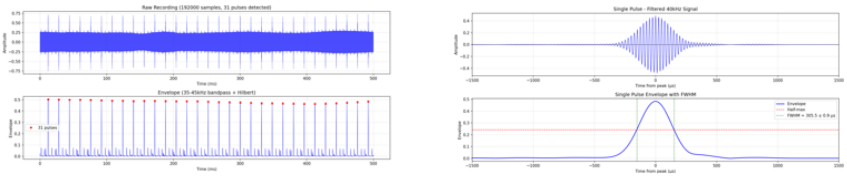




Hmm those are not really Gaussian envelopes are they. Well the sample rate is all wrong--for a pulse frequency of 62.5Hz, we want the sample rate to be 400kSa/s. But that still didn't change the overall envelope shape.

After some more debugging with Claude, we discovered **the issue was a byte order (endianness) bug in the waveform data encoding.** When computers store multi-byte numbers (like 16-bit integers), they must choose which byte to store first. "Big-endian" stores the most significant byte first, and vice versa for "little-endian". The SDG1062X expects waveform data in little-endian format (weird), but the original code was sending big-endian data. This meant a value like 8191 (hex1FFF) was being sent as bytes 1F FF, but the SDG interpreted it as FF 1F = -225. Every sample in the waveform was being scrambled to a completely wrong value, which is why the output looked like noise rather than a clean Gaussian envelope. Code is updated above.

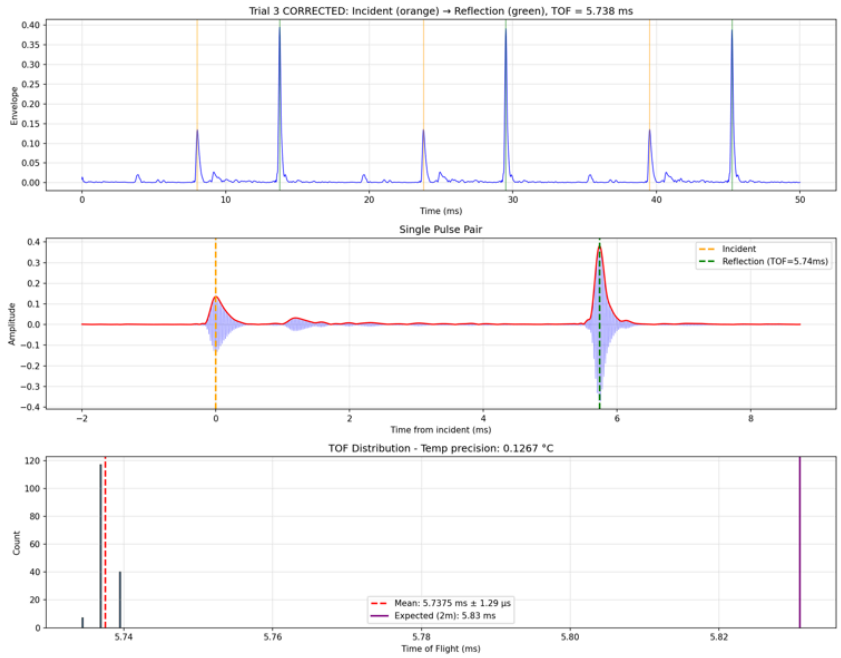
Updated gaussian trial (as picked up by mic):



From the bottom left figure, it seems like the microphone is still picking up some multipath (>1 small peak per period).

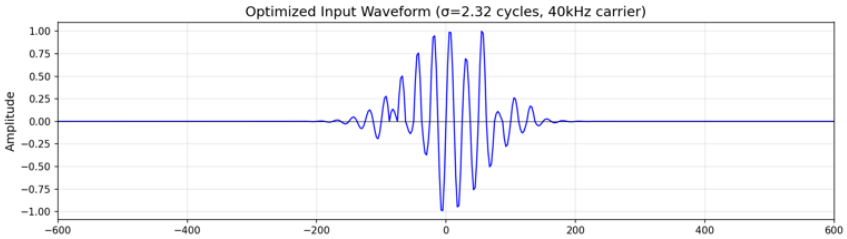
New Optimization (June 11)

Playing around with the positioning of the mic relative to the piezo speaker array. I had previously mounted the mic ~8cm away from the center of the piezo array, which could be contributing to loss of signal, especially in the noisy environment of the lab (25kHz noise is still there). Repositioning the mic to sit right next to the piezo speaker worked well:



Recording and Results from sending optimized_waveform_v1 pulses for 0.5s. Expected time of flight (TOF) is based on ideal environment conditions set at the temperature read off of the thermometer (~66 F in the lab), as well as perfect 2m path length. Uncertainty on path length measurement is increased in the lab setup due to the usage of the corner-cube reflector, so the offset of experimental results from expected TOF is not significant

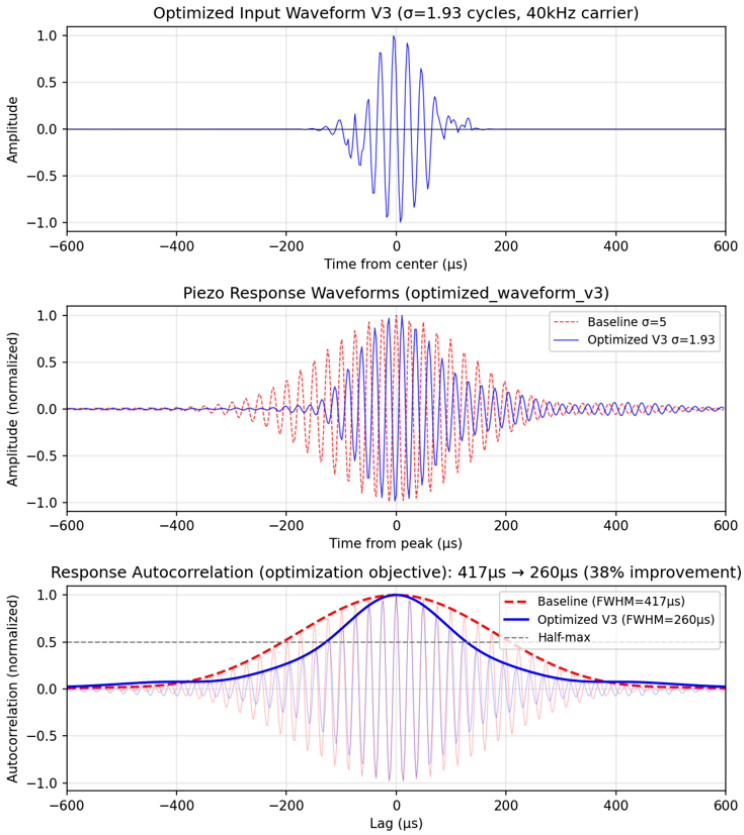
I am running a new round of Bayesian Optimization where the objective function is now the narrowness of the ACF peak. Note that the autocorrelation FWHM is essentially twice the pulse envelope FWHM (since autocorr of a Gaussian is a wider Gaussian). For 200 iterations, it gave the following, which we will call **optimized_waveform_v2**.



optimized_waveform_v2

See the **Bayesian Optimization v2** page (above) for more information about this result.

Since this looked quite similar to **optimized_waveform_v1**, I thought I would up the number of iterations in the Machine Learning algorithm to 1000, producing the following (**optimized_waveform_v3** info is here):



Still, not too different.

Limitations of the gaussian+compensation Parametrization

optimized_waveform_v2 gave a 31% improvement in the FWHM of the Auto-Correlation Function (ACF) peak (443-->307μs), and **optimized_waveform_v3** gave a 38% improvement in the FWHM of the ACF peal (443⟶275μs). These gains are significant, but the marginal increase of 7% improvement with the extra 800 iteration tells me that Bayesian Optimization under this parametrization is meeting some barrier preventing improvement past ~40%. This barrier could potentially be the theoretical limit of the optimization--there is a case to be made that the ringdown of the piezo transducers is a physical effect that, without targeted inverse filtering, will always spread the energy of the signal. However, I think that a first-order contributor to this barrier is the current parametrization. Under the first three rounds of optimization (v1, v2, v3), the input waveform was modeled as a modifiable gaussian with K-many “compensation” functions:

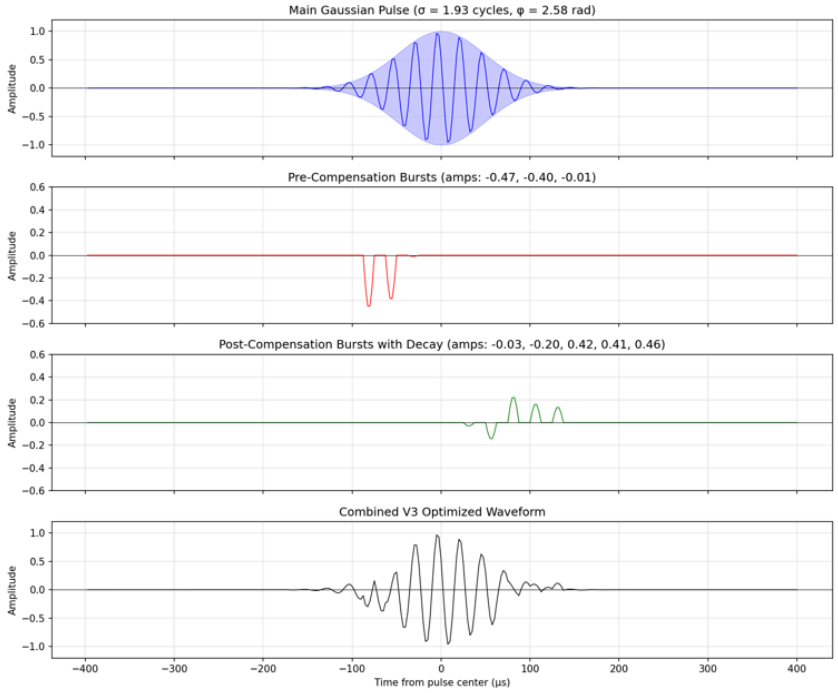
Mosaic macros cannot be exported to PDF.

These “compensation functions” were designed to be pre-pulse or post-pulse sinusoidal bursts at the carrier frequency, with the goal being to have extra coverage for the time intervals when the behavior of the piezo could be most nonlinear (i.e. before or after the pulse). So, each carrier function g_k is either a pre-compensation or a post-compensation.

1. **Pre-compensation:** There were 3 half-cycle bursts placed at fixed intervals before the main pulse center, each taking the form amp * sin(2π f_c t_burst) for its rectangular window.
2. **Post-compensation:** There were 5 half-cycle bursts placed at fixed intervals after the main pulse center, each taking the form amp * exp(-i * 0.3) * sin(2π f_c t_burst). The exponential decay is included to mimic the ringdown.

The optimization waveforms made some use of these compensation functions. The amplitudes of the three pre-center bursts were [-0.47, -0.40. -0.01] (relative to the waveform maximum). The amplitudes of the five post-center bursts

were [-0.03, -0.20, 0.42, 0.41, 0.46].

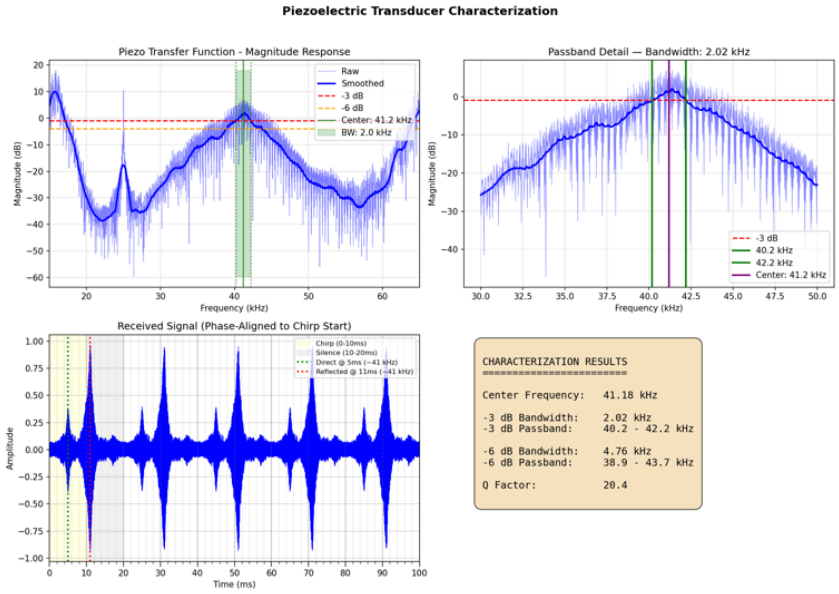


Due to the parametrization, *the compensation functions are the only source of deviation from a strict gaussian.* In this way, the first three rounds of optimization have been very restricting, leaving simple sinusoidal bursts (at pre-set positions) as the only means of deviating from the gaussian structure. This leaves an open question of how to parametrize the waveform to yield a more expansive optimization.

Here's what Claude has to say:

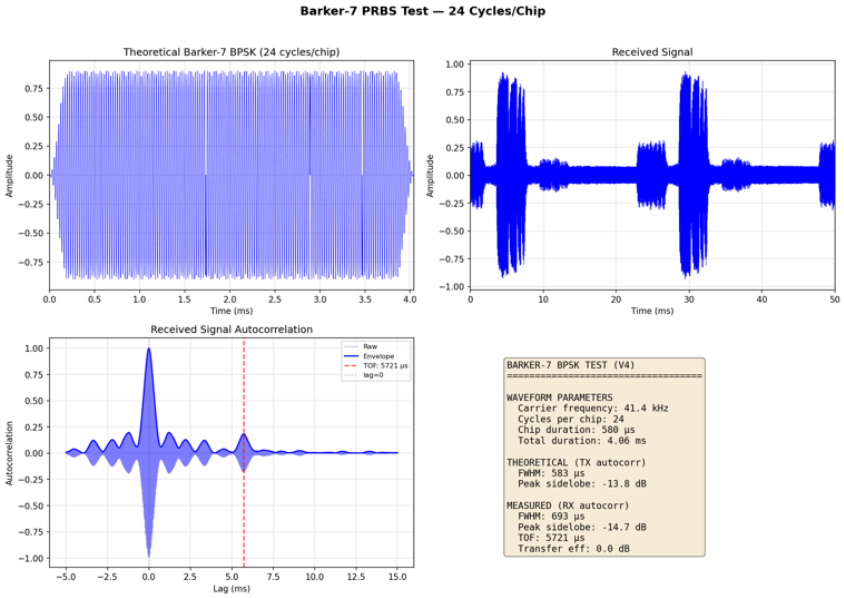
> "The optimizer essentially discovered a crude form of mismatched filtering or sidelobe suppression - using out-of-phase components to destructively interfere with the autocorrelation sidelobes. This is the same principle behind Barker codes and NLFM, just discovered empirically within a more constrained parameterization."

Piezoelectric Transducer Bandwidth Characterization

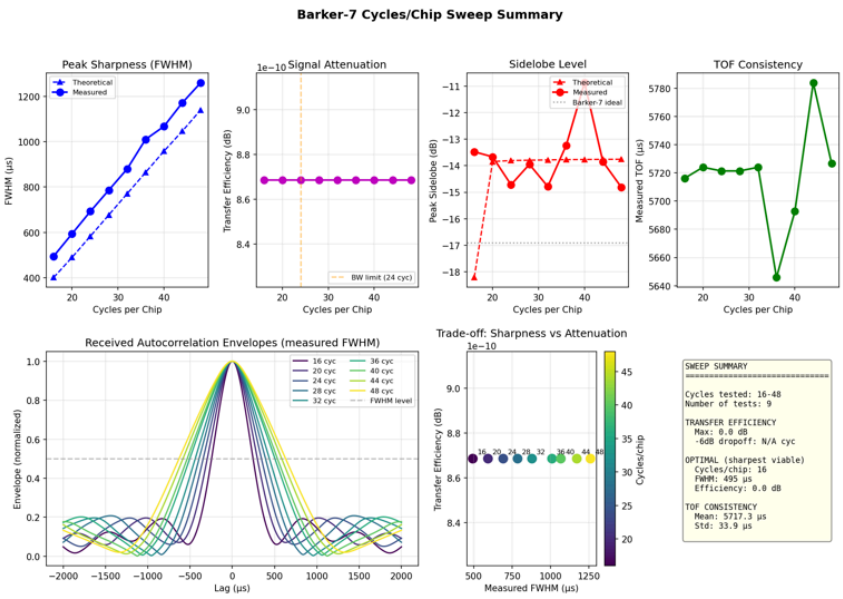


Exploring Binary Pseudo-Random Sequences (BPRS) as an alternative waveform

Barker-7 Code for TOF Calculation (barker7-bprs-tof-testing-v4)



Top Left: computer-generated waveform consisting of Barker-7 modulated onto 41.2kHz carrier sine wave (using binary phase shift keying), i.e. waveform uploaded to Siglent SDG; **Top Right:** piezo response waveform, as received by the Dodotronic microphone; **Bottom Left:** autocorrelation of a single period of piezo response to Barker-7; **Bottom Right:** parameter info



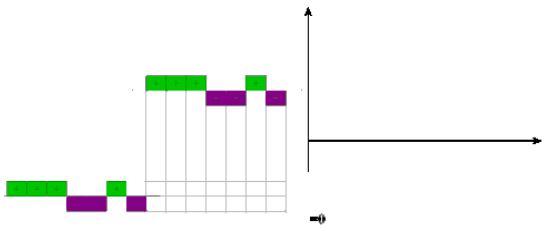
This is for the Barker-7 Code (+1, +1, +1, 0, 0, +1, 0), with number of cycles per chip (bit of the Code) from the set {16, 20, 24, 28, 32, 36, 40, 44, 48}. Zip file with the panels of plots (input waveform, piezo response, autocorrelation, parameter info) for each cycle/chip value is attached below

barker7_v4.zip
18 Jun 2026, 01:56 AM

Still some funky business with the signal attenuation:

> I expected attenuation of the signal when the barker code has less than 24 cycles per chip, because when the chip duration is that low (chip duration T_C = cycles per chip / 41.2kHz = 24/41,200 =0.000583 s), then the bandwidth of the signal (B_s ~ 1/T_c = 1715 Hz) is larger than the characteristic bandwidth of the piezo array (as measured to be ~1.7kHz), and we would start to have attenuation of the high-frequency components of the signal. This would smear the auto-correlation function peak. However, as the little attenuation plot shows, all values for cycles/chip have the same transfer efficiency. This doesn't make sense to me.

The cleanest result (biggest victory) seems to be the FWHM of the autocorrelation function peaks. The top left plot is also informative, showing autocorrelation peak FWHM increasing linearly with the number of cycles per chip. This is due to the fundamental property that of the Barker Codes: the codes are sequences of bits with the unique property that the autocorrelation peaks at lag-0 with amplitude = #chips, and takes a value in {-1, 0, 1} at all non-zero lags. This can be seen in the following gif:



Barker-7 Code Autocorrelation, courtesy of radartutorial.eu

When we modulate the binary sequence onto the phase of the carrier wave, each chip (bit) is assigned an equal time duration of the signal that it occupies (this is T_c). This means that the FWHM of the lag-0 peak is limited by the chip length T_c that we choose for the waveform (because the autocorrelation will shift out of peak alignment at the very latest once the correlation lag hits the next T_c interval, i.e. when the next phase shift happens). Essentially, T_c sets how fast the code decorrelates; in the case of the Barker-7 code, the peak goes from 7 to 0 at lag=T_c, so the half-max crossing (~3.5) occurs somewhere within that interval. Thus, FWHM $\propto$ T_c.

Then, the relationship between FWHM and the number of cycles per chip (N_cyc) follows trivially, as T_c is proportional to N_cyc, just scaled by the carrier frequency, which is kept fixed at 41.35 kHz. In other words, if FWHM $\propto$ T_c = N_cyc / f_carrier, then FWHM scales linearly with N_cyc. This is exactly what we observe in the “FWHM vs. cycles per chip” graph in the top left of the figure above.

Waveform Performance Analysis, and chirped waveform testing

Waveform	Recording Date	Filter	FWHM (µs)	FWHM σ (µs)	TOF (µs)	TOF σ (µs)	Ratio to lag-0 peak (dB)	N
optimized_gaussian_v2	June 11	hp30kHzord8	292	0	5723.1	4.6	-11.3	29
optimized_gaussian_v3	June 11	hp30kHzord8	250	0	5741.0	0.6	-11.4	30
chirp_lfm	June 19	no filter	292	0	5678.9	3.3	-20.5	18
barker7_16cyc	June 18	no filter	495	0	5725.4	0.5	-12.1	18
barker7_24cyc	June 18	no filter	692	2	5733.8	1.8	-13.2	18

Narrative on FWHM_corr, σ_TOF, SNR_corr, and δ_FWHM

Definitions:

1. FWHM_corr: the full width at half max of the time-of-flight peak in the auto/cross-correlation plot of a single period of pulse travel; it is calculated by identifying the arg_max of the peak region, and then running a parabolic fit on [arg_max - 5, arg_max + 5] (i.e. spanning from the 5 points before the crude max to the five points after it); the point on the fitted parabola at which its slope is 0 is the returned FWHM_corr
2. δ_FWHM: the uncertainty placed on the FWHM_corr determination by the fitting algorithm; originally, when a three-point parabolic interpolation was used, the system was fully constrained (#degrees of freedom = 0), and therefore there was no uncertainty to return; however, now, δ_FWHM gives a meaningful statistical metric for the per-period peak determination that can be compared with between-period metrics, like σ_TOF.
3. σ_TOF: the standard deviation from the mean time-of-flight measurement, averaged over consecutive independent pulses

As a metric to compare different waveforms during the optimization process, I have been using FWHM_corr to understand the distinct shape and theoretical narrowness that different waveforms can achieve. FWHM_corr is a robust metric, and it serves a particular purpose; however, it is not a direct determinant of the temperature resolution achievable in BATPING's final implementation. Alternatively, σ_TOF is the metric that ultimately determines temperature sensitivity, and, unlike FWHM_corr, it is sensitive to inter-measurement systematic and statistical error (more below).

Why FWHM_corr is right for its current usage:

While in the end we would like to know the precision with which we can detect temperature fluctuations (i.e. σ_TOF), the current objective is to design a waveform optimized for the piezoelectric transducer's non-linear response that affords the most accurate TOF calculations under standard conditions. FWHM_corr is a robust method of measuring waveform performance, because it is less influenced by the variation of statistical/systematic noise between successive pulses. σ_TOF is fundamentally a statistic representing multiple pulses, and it comes with its own error that scales like 1/√N (σ_{σ_TOF} = σ_TOF/√N). FWHM_corr, alternatively, is a single-pulse measurement observed to have very little variation between pulses. An input waveform will reliably elicit the same piezo response behavior (with some variable noise introduced along the way by the waveform generator, wires, air and audio disturbance, microphone noise floor, etc). This means that the peaks in the autocorrelation are expected to look the same, even if the location of the peaks may vary according to σ_TOF.

Why FWHM_corr is limited in predicting final temperature precision:

The FWHMs of peaks in an autocorrelation plot measure correlation peak width under fixed conditions: same temperature, same path length, same environment. Because σ_TOF measures between-measurement variation (rather than the uncertainty of the time-of-flight determination for a single ping), σ_TOF depends jointly on FWHM_corr and SNR:

Mosaic macros cannot be exported to PDF.

A narrower peak with poor SNR can actually give a *worse* calculated σ_TOF in the end, as compared to a wider peak with a high SNR. The chirp compression experiments below demonstrate this quantitatively: the most aggressively compensated waveforms achieved FWHM = 98µs (much sharper than the optimized Gaussian V3 at 250 µs) *but actually gave higher* σ_TOF, *potentially because the compensation amplifies noise at the spectral edges as much as it does the signal.* While I expect FWHM (or rather FWHM/SNR) to track with σ_TOF, it is very possible that an elevated noise floor meant that this trend gets drowned out by noise.

insert figure here showing expected and measured FWHM/SNR vs σ_TOF (with analysis)

Regardless, the cleanest metric for final performance is σ_TOF. For reference, σ_TOF is converted to tempreature precision using:

Mosaic macros cannot be exported to PDF.

This means that our target of 0.1 °*C* is achieved by a σ_TOF < 1.02 µs per ping.

LFM Chirped Waveforms

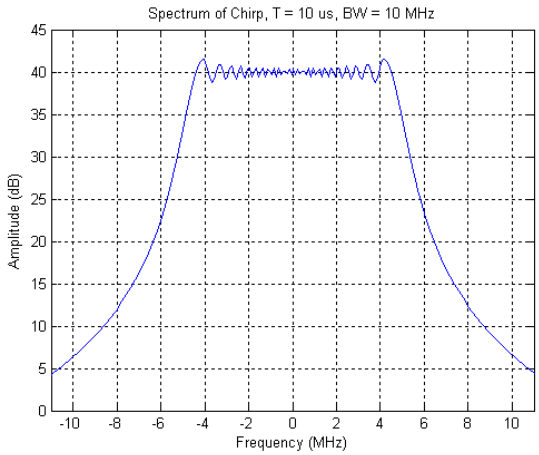
A linear frequency-modulated (LFM) chirp sweeps from f_start to f_stop over duration T, producing a nearly-flat input signal power spectrum over bandwidth B = f_stop - f_start. Professor Stubbs aptly described this as a band-limited version of a delta function's power spectrum, *spread out in time.* While a delta function contains all frequencies simultaneously, the chirp contains them sequentially. Given that our initial motive was to use a waveform that illicited a delta function piezo response (by eliminating ringdown tail and narrowing the response body), I tested the delta spectrum-like linear chirp.

Chirp Cross-Correlation Peak Shape, and Expected FWHM

An LFM Chirp with bandwidth B has an approximately flat power spectrum over [f_0 - B/2, f_0 + B/2]:

Mosaic macros cannot be exported to PDF.

When the Time-Bandwidth product of the chirp is very large, then this approximation works very well. Take for example the power spectrum of this 10 us, 10 MHz chirp that I found online, which has TBP=100:



The Wiener-Khinchin Theorem states that the power spectral density is equal to the Fourier Transform of the autocorrelation. This gives us a sense for the theoretical autocorrelation peak shape. Reframed, to get the autocorrelation shape y(τ), simply take IFT(PSD):

Mosaic macros cannot be exported to PDF.

because the Fourier Transform of the sinc is a rectangular window, and vice versa. **So, we expect the chirped waveforms to have sinc-like correlation plots.**

From this, we can calculate the expected FWHM for a sinc-like correlation peak. We want to take the FWHM of the *power response*, not amplitude, of the correlation plot. The half-max of sinc^2 is:

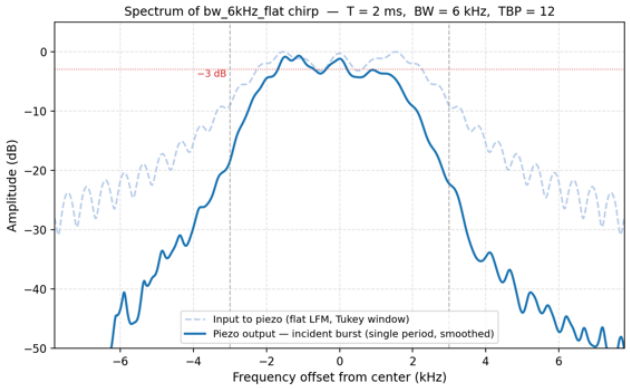
> sinc^2(Bτ) = 0.5 → sinc(Bτ) ~ 0.707 → Bτ ~ 0.443
>
> FWHM = 2 x 0.443 / B = 0.886 / B

So, we expect the -3dB power width of the matched filter output to be 0.886/B wide.

So, for the piezo's -3dB bandwidth of 1.7 kHz, FWHM ~ 521 µs. To determine whther this FWHM could be narrowed, I performed a sweep of different chirp bandwidths that surpass the -3dB range. As will be seen below, normal chirped waveforms with larger bandwidths, while expected to have lower FWHM_corr, will not perform well due to the high-Q nature of the piezo arrays. A variety of amplitude compensation methods have been tested to account for the loss of signal when driving the piezo far off-resonance.

Caveat in Chirp Power Spectrum

In reality, however, when TBP is smaller and when the chirped signal passes through the piezo array and is acted on by a linear operator, the output spectrum will not be perfectly rectangular. For example:



Power Spectra of input and output waveforms, when a 6kHz, 2ms chirp was sent into the piezo array

Here, the piezo output power spectrum does nicely match the divots on the flat top of the input waveform spectrum, but it is narrower and more ‘peak-y’.

However, the main point is that exploration of chirp duration, bandwidth, and amplitude shape is warranted by intuition about chirp fidelity and preliminary results.

Chirped Waveform Testing

Below is a guide to the different characteristics of the chirped waveform that were modified, and the corresponding keys for each waveform:

Bandwidth/frequency range:

- "bw_6kHz" -- chirp bandwidth = 6 kHz, centered symmetrically around resonance (f_0 ~ 41.5 kHz). This is pretty narrowband.
- "bw_20kHz" -- chirp bandwidth = 20 kHz (31.5-51.5 kHz). This is pretty wideband.
- "bw_25kHz" -- chirp bandwidth = 25 kHz (28 - 54 kHz). This is the widest configuration.
- "hf_m15dB_asym" -- "high frequency asymmetric". The passband is chosen to include all frequencies above the -15dB cutoff. This results in a passband that is not symmetric about f_0, because the piezo resonance (as measured by the transfer function during my piezo characterization) extends further on the low-frequency side.

Amplitude shaping:

- "comp" -- chirp amplitude = 1/H(f), where H(f) is the transfer function measured during piezo characterization. Ideally, amplitude-compensated waveforms should have a flat acoustic output spectrum because they compensate for the fact that the piezo will resonate less well when you drive it off-resonance. This has a 10% floor, meaning that it will stop boosting frequencies when they are lower than the -20dB point in the transfer function.
- "comp_light" -- same amplitude compensation as above, but with a 20% floor (max -14dB boost). This was supposed to be a less aggressive form of amplitude compensation.
- "diff" -- diffraction-compensated: applies an additional f/f_0 factor on top of the 1/H(f). I did this because Professor Stubbs suggested that the weird "forked" peak shape we were seeing could have been because of diffraction determining the arrival time of each frequency's beam of signal.
- "flat" -- no amplitude compensation. Honestly, since it performs as well as the amplitude-compensated waveforms, maybe we should stick with it since it's so simple.
- "comp_rect" -- same compensation as "comp" but without the default Tukey window (Tukey window is essentially just a sine build-up and build-down in amplitude at the start and end of the chirp)

Duration:

- 2ms, 4ms -- chirp duration before dead time. Longer duration should mean a larger time-bandwidth product, which would produce higher gain and a sharper autocorrelation peak in theory, though I'm not observing this currently.

Alpha Sweep (alpha_aX.X_bw_dur):

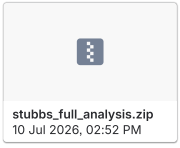
- "a0.2" ... "a0.8" -- Another method of lightening the amplitude compensation: exponent α in H(f)^{−α}. α=0 is flat (no compensation), α=1 is full 1/H(f).
- "sm" -- small bandwidth (6kHz, 38.5-44.5 kHz)
- "md" -- medium bandwidth (12 kHz, 36-48 kHz)
- "lg" -- large bandwidth (20 kHz, 32-52 kHz)

A Two-Template Matched Filtering Method: The Stubbs Pipeline

The Stubbs pipeline

- Record ~400 periods. The first 100 are template periods (periods that will be used to construct the templates), and the remaining are evaluation periods (periods that will be treated as signals in the matched filtering).
- Co-add masked versions of the first 100 pulses to create two templates: (1) the incident peak template is the co-adding of periods where the waveform has been masked around the incident pickup, and (2) the reflection peak template is the co-adding of periods where the waveform has been masked around the reflected signal. Proper construction of these templates required precise identification of each ping's period. With data where recording times were >10s, this meant accounting for significant drift in Ultrasonic mic + Siglent Waveform Generator. First, a crude determination of peak location was performed by finding the global max in each expected period range. Seeing as I have re-oriented the microphone such that incident pickup and reflection signals are recorded with roughly equal amplitudes, global max could be in either peak. So, I set a peak-finding algorithm to search for peaks at ±TOF (±5.7ms) before and after t_{global max}. After finding both peaks, I set the window to be 2ms before the first peak maximum. As mentioned above, this is only a crude period construction, because there will be jitter between the actual start of each period and when the incident pickup amplitude is maximized. So, for every template period, run a **cross-correlation with the first template period (per0)**. The large central peak lag (hopefully near 0, but not guaranteed to be exactly 0) will give the window correction needed to properly align the period with per0.
- Perform matched filtering with the two templates on all evaluation periods. Δτ = τ_reflection - τ_launch gives the differential TOF relative to the environmental conditions when the template was created.

All of the first-round recordings (completely V0--no corrections made) from July 9 of characteristic chirped waveforms and alpha-sweep waveforms can be found in this zip file:



It was not easy to get this pipeline working. Claude struggled with peak-finding and proper windowing around subsequent periods (not for the first time...) while accounting for timing jitter and drift of Siglent SDG waveform generator and the Dodotronic ultramic. My understanding of the main problem encountered is as follows:

> While our ultrasonic transducers use piezo crystals to as a physical converter between mechanical and electrical signals, they are often implemented as internal clocks for electronics equipment and hardware. In our case, both

the SDG1062X and the Ultramic 384K have thier own internal crystal oscillator (probably quart); neither clock knows the other exists. The SDG uses its oscillator to time the burst repitition, staying true (with jitter) to the rep rate programmed into the waveform.

For example, a 7600-sample waveform sent into the Siglent's waveform buffer with a programmed readout rate of 400 kSa/s will produce a total signal + dead time duration of 7600 * 400000 = 19ms. This is precisely the computation that can be found in any code that performs waveform upload. In this particular case, this is the total duration of chirped waveforms sent with chirp_dur = 4ms, dead_dur = 15ms. The rep rate is also easily derived: rep_rate = 400 kSa/s / 7600 samples = 52.63 Hz.

The Ultramic similarly uses its own clock to regulate its analog-to-digital conversion of the signal, which occurs at the advertised rate of 384 kHz. Claude says that consumer/lab-grade crystals drift at 50-300 ppm from their nominal frequency, and that combining two independent oscillators will produce a *relative* drift of up to ~200-400ppm. For a 19ms total period:

- nominal UltraMIc samples per period = 384 kHz * 19 ms = 7296 samples/period
- **measured** drift rate (measured here) = 4.7 us/period * 384 kSamples/10^6 us = 1.8 samples/period
- inferred relative clock offset (in parts per million) = 1.8 / 7296 = **246 ppm**

This is significant. It means that if we record 100 periods, the acumulated drift will be 180 samples, or 469 us. If we record 30s of periods (at 19ms, this is ~N=1478 period), then the accumulated drift is in the **milliseconds**: 2670 samples, or equivalently, 6.95ms.

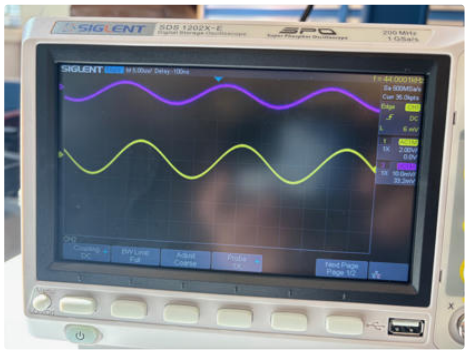
New Microphone Repeatability Measurements

We are switching from the Dodotronic Ultramic, which digitizes at 384kHz, to the Superlux ECM999 *analog* measurement microphone. The ECM999 requires a 48V phantom power box, as well as a preamplifier (the one we found in the lab is an SRS Model SR560 low-noise preamp). The whole configuration looks something like this:



ECM999 Testing setup, with important bench-top electronics/audio components circled.

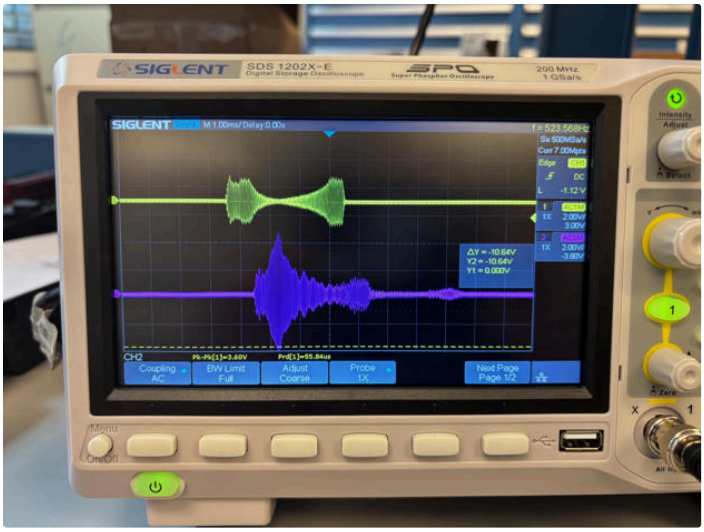
A T-connector sends one BNC cable from the Siglent WG to the piezo array, and the other to **channel 1** on the scope (the input waveform is sent to channel 1 so that we can easily trigger on it). The mic is placed directly in the piezo array's acoustic beam for first-step signal verification. The mic is fed into the preamp using XLR-XLR cable, and an XLR-BNC lead goes into the input of the SR560 preamp. The preamp gain is set to 100 when a signal is being output by the Siglent, and 1 when no signal is being sent (so as to avoid overload). The output of the SR560 goes into **channel 2** on the scope. Here is a simple 40kHz sine wave successfully sent from the WG with Professor Stubbs' help on Sat. Jul 12:



And here is an alpha-compensated chirped waveform, with parameter α=0.8, bandwidth 20kHz, and duration 4ms

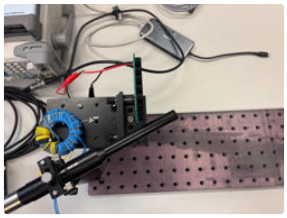


Waveform info from Siglent WG

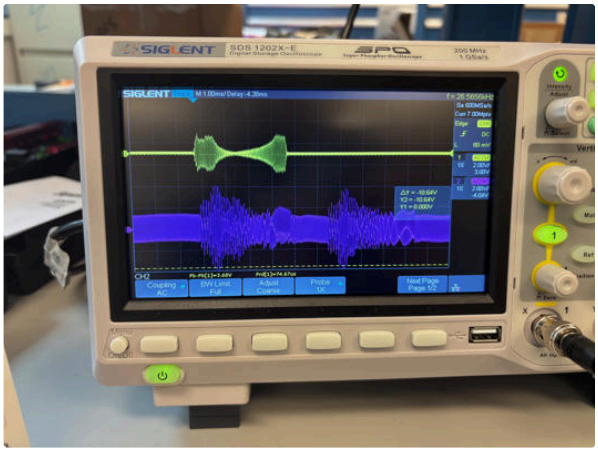


Oscilloscope traces (yellow is input, purple is mic output) with preamp G=100, time scale of 1ms/div, and both channels amplitude scale of 2V/div

Okay, now that we've verified that we can pick up waveforms broadcasted with the piezo array using the new mic (which is more than what we could say for the more expensive Earthworks M50 measurement mic...), let's reconfigure the reflection setup, and fine tune the positioning of the microphone so that incident and reflection pickup are nearly equal in amplitude. This positioning does it, though I don't know why the output trace (channel 2) is so noisy:



Mic positioning that produces equal incident and reflected pickup amplitude. We may worry about the mic blocking 1-2 transducers (this probably won't affect signal power so much, but the reflection back into the piezo array might matter).



Scope trace with incident and reflection piclup equal in amplitude. The noise in output is odd. G-500 on the preamp.

The pre-amp amplifies signal and noise alike, so the raggedy scope trace could simply be the result of the (1) limited omnidirectionality of the microphone (info sheet below in Files) causing incident pickup to be limited, and (2) radar equation limiting reflected pickup, especially at the non-zero mic angle. Let's test with the Picoscope USB scope, which samples at rates up to 1 GSa/s! This is the advantage of switching to the analog mic--we are no longer limited by the digitization rate of the Ultramic.

Files

Hardware Data Sheets:

Ultramic384_EVO_User_Guide.pdf

datasheet_murata_ma40S4S-R.pdf (specs for the piezo transducers we're using? update: we might end up using 10mm or 18mm transducers from CUI)

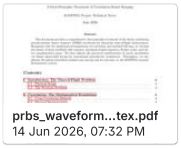
cusa-t80-12-2200-th.pdf (these are the correct data sheets; CUI is now Same Sky)

superlux_ecm999_infosheet.pdf

picoscope-5000d-series-users-guide.pdf

Reports, Informational Write-ups:


cyrusurheim_B... sal.pdf
27 Mar 2026, 06:48 PM


prbs_waveform...tex.pdf
14 Jun 2026, 07:32 PM